

UNIVERSIDAD DE LAS TUNAS
FACULTAD DE CIENCIAS TÉCNICAS Y AGROPECUARIAS
DEPARTAMENTO DE INFORMÁTICA

Sistema de gestión de menú en los comedores de la Universidad de Las Tunas

TRABAJO DE DIPLOMA EN OPCIÓN AL TÍTULO DE INGENIERO INFORMÁTICO

AUTORA: Yamilet Yero Hechavarría

TUTORA: Ing. Annalie González Rodríguez

Las Tunas, de junio del año 2025

PENSAMIENTO

"Sabiduría ante todo; adquiere sabiduría; y sobre todas tus posesiones adquiere inteligencia."

Proverbios 4:7 (Reina-Valera 1960)

AGRADECIMIENTO

Anexo 5. PLANTILLA PARA DECLARACIÓN DE AUTORÍA

Yo Yamilet Yero Hechavarría, declaro que soy el único autor del trabajo Sistema de gestión de menú en los comedores de la Universidad de Las Tunas y autorizo a la Facultad de Ciencias Técnicas y Agropecuarias, así como al Departamento de Informática para que hagan el uso que estimen pertinente con este trabajo.

Para que así conste firmo la presente a los ____ días del mes de _____ de _____.

Firma del Autor

Nombre, apellidos y firma del Tutor

Anexo 6. PLANTILLA PARA OPINIÓN DEL USUARIO DEL TRABAJO DE DIPLOMA

OPINIÓN DEL USUARIO DEL TRABAJO DE DIPLOMA

El Trabajo de Diploma, titulado <título>, fue realizado en <lugar>. Esta entidad considera que, en correspondencia con los objetivos trazados, el trabajo realizado le satisface

- Totalmente
- Parcialmente en un ____ %

Los resultados de este Trabajo de Diploma le reportan a esta entidad los beneficios siguientes:

Y para que así conste, se firma la presente a los ____ días del mes de _____ del año ____

Representante de la entidad

Cargo

Firma

cuño

RESUMEN

La investigación se enfoca en solucionar las deficiencias presentadas en la gestión de menú en los comedores de la Universidad de Las Tunas. Actualmente, el proceso se desarrolla de forma manual e involucra a múltiples personas, lo que propicia la existencia de retrasos en el extenso procedimiento de elaboración de los menús. Asimismo, la reserva de tickets para el comedor exige la presencia física obligatoria de los interesados en las instalaciones universitarias. La gestión de reportes se efectúa manualmente, requiriendo una inversión considerable de tiempo debido al amplio volumen de información y la frecuencia con que estos se generan. Adicionalmente, el sistema actual no ofrece la posibilidad de realizar reservaciones ni efectuar pagos por medios electrónicos. Por consiguiente, el actual trabajo de tesis se ha orientado al desarrollo de un sistema informático destinado a la gestión de los menús de alimentación en la Universidad de Las Tunas, con el fin de erradicar las deficiencias previamente descritas. Para su implementación, se han empleado diversas tecnologías, incluyendo: Spring Boot 3 como framework para el desarrollo del servidor con Java, Flutter y Dart para la construcción de la interfaz de cliente, PostgreSQL como sistema gestor de base de datos y la metodología Scrum para la conducción del proyecto.

Palabras clave: sistema informático, gestión de menú.

ABSTRACT

This research focuses on addressing the deficiencies present in menu management within the dining halls of the University of Las Tunas. Currently, the process is conducted manually and involves multiple individuals, leading to delays in the extensive menu preparation procedure. Furthermore, reserving dining hall tickets necessitates the mandatory physical presence of interested parties at the university facilities. Report management is performed manually, requiring a considerable time investment due to the large volume of information and the frequency of their generation. Additionally, the current system does not offer the possibility of making reservations or processing payments through electronic means. Consequently, the present thesis work is oriented towards the development of a software system for the management of food menus at the University of Las Tunas, aiming to eradicate the previously described deficiencies. For its implementation, diverse technologies have been employed, including: Spring Boot 3 as the server-side development framework with Java; Flutter and Dart for the construction of the client interface; PostgreSQL as the database management system; and the Scrum methodology for project management.

Keywords: software system, menu management.

ÍNDICE

INTRODUCCIÓN.....	10
CAPÍTULO I. MARCO TEÓRICO REFERENCIAL.....	15
1.1 Gestión de la alimentación.....	15
1.1.1 Gestión del menú de alimentación en las universidades.....	16
1.1.2 Proceso de gestión de la alimentación en la Universidad de Las Tunas....	17
1.2 Soluciones de software en la gestión del menú de alimentación.....	18
1.3 Metodologías, tecnologías, lenguajes y herramientas.....	21
1.3.1 Metodologías.....	22
1.3.2 Frameworks.....	25
1.3.3 Lenguajes de programación.....	29
1.3.4 Sistemas gestores de Base de Datos.....	33
1.3.5 Patrones de diseño y arquitectura.....	34
1.3.6 Herramientas.....	36
Conclusiones del capítulo.....	39
CAPÍTULO 2. DESCRIPCIÓN Y CONSTRUCCIÓN DE LA SOLUCIÓN PROPUESTA.....	40
2.1 Fase de Iniciación.....	40
2.1.2 Product Backlog.....	41
2.1.3 Estimación total del proyecto.....	44
2.1.4 Definición de Roles y Funcionalidades.....	44
2.1.5 Historias de Usuario.....	45
2.1.6 Restricciones que el sistema debe cumplir.....	49

2.2 Fase de Planificación.....	52
2.3 Diseño de la base de datos.....	55
2.4 Codificación y Pruebas.....	56
2.4.1 Codificación.....	57
2.4.2 Estándar de codificación.....	58
2.4.3 Pruebas.....	59
Conclusiones del capítulo.....	68
CAPÍTULO III. IMPLEMENTACIÓN DEL SISTEMA.....	69
3.1 Diseño de las interfaces de usuario.....	69
3.2. Protección y seguridad del sistema.....	73
3.2.1 Distribución física del sistema.....	75
3.3 Forma general y principios en que se basa el sistema de ayuda.....	76
3.4 Tratamiento de errores.....	76
3.5 Importancia del sistema.....	77
RECOMENDACIONES.....	79
REFERENCIAS BIBLIOGRÁFICAS.....	80
ANEXOS.....	83

INTRODUCCIÓN

A medida que avanzamos en el siglo XXI, las universidades cubanas se encuentran en la vanguardia de una transformación sin precedentes en el ámbito de la gestión de servicios alimentarios. En un mundo cada vez más interconectado y digitalizado, los comedores universitarios deben adaptarse a las nuevas realidades de la conectividad global, la innovación constante y la transformación digital. Estos espacios, cruciales para el bienestar de la comunidad universitaria, enfrentan desafíos que reflejan la necesidad de una evolución constante para satisfacer las expectativas nutricionales y culinarias de una población diversa [1].

La elaboración de menús diferenciados en la universidad es un claro ejemplo de la innovación constante requerida para mantenerse al día con las demandas nutricionales y culinarias. La gestión de estos servicios alimentarios debe superar los desafíos de la sostenibilidad ambiental, asegurando que la calidad y la variedad no se vean comprometidas por la limitada disponibilidad de productos alimenticios. La necesidad de adaptar los menús a las fluctuaciones en la entrega de suministros requiere una planificación flexible y creativa, lo que a menudo resulta en una tarea compleja y en constante cambio.

La implementación de un sistema de gestión de menús digitalizado puede ser la clave para superar los obstáculos presentes en la logística de los servicios de alimentación. Estas herramientas digitales, fundamentales en la interconexión global, ofrecen una oportunidad para mejorar la eficiencia operativa y la experiencia del usuario en los comedores universitarios.

Los sistemas de gestión de menús digitalizados en los comedores universitarios representan una revolución en la forma en que las instituciones educativas abordan la nutrición y la satisfacción de sus estudiantes y personal. Este sistema es una plataforma integral que permite a los administradores de comedores actualizar y modificar menús, responder a la disponibilidad de ingredientes y adaptarse a las

preferencias dietéticas con una eficiencia sin precedentes. Con interfaces de usuario intuitivas, estos sistemas pueden ofrecer análisis detallados como los costos de ingredientes y las tendencias de consumo, lo que permite una planificación de menús más informada y orientada a datos.

Los códigos QR (acrónimo de Quick Response) son un tipo de imagen de código de barras que se puede escanear con un teléfono inteligente y almacenar información [2].

El uso de códigos QR para el acceso a los menús y la reserva de tickets es otro avance significativo. Cuando se escanean con un dispositivo móvil, pueden dirigir a los usuarios a menús digitales actualizados, permitiendo a los comensales tomar decisiones informadas sobre su alimentación antes de llegar al comedor. Además, los códigos QR pueden ser utilizados para realizar pagos sin contacto, lo que no solo acelera el proceso de transacción, sino que también mejora la seguridad al reducir la necesidad de manejar efectivo y limitar las interacciones físicas, un aspecto particularmente importante en el contexto de la salud pública [3].

Estas herramientas digitales son esenciales en la interconexión global, ya que no solo mejoran la eficiencia operativa y la experiencia del usuario en los comedores universitarios, sino que también se alinean con las tendencias globales hacia la digitalización y la automatización. Al adoptar estas tecnologías, las universidades pueden garantizar que sus servicios de alimentación sean accesibles, higiénicos y atractivos para la comunidad universitaria, al tiempo que se posicionan como líderes en la innovación de servicios estudiantiles. Además, la implementación de estas soluciones digitales puede servir como un modelo para otras instituciones que buscan modernizar sus operaciones y mejorar la calidad de vida de sus comunidades.

En la Universidad de Las Tunas, los desafíos en la gestión de los servicios de alimentación se manifiestan de varias maneras, impactando significativamente la

eficiencia y la satisfacción de los usuarios. La gestión manual de los menús y la falta de un sistema de inventario digitalizado han llevado a dificultades en la adaptación a la variabilidad de los suministros alimenticios, resultando en una oferta limitada y a veces monótona que no cumple con las expectativas ni las necesidades nutricionales de la comunidad universitaria.

Estos problemas no solo afectan la operatividad diaria de los comedores, sino que también tienen un impacto en la percepción de los servicios por parte de los usuarios. La incapacidad de adaptarse rápidamente a los cambios puede disuadir a los estudiantes y trabajadores de utilizar los comedores, optando por alternativas externas que podrían ser más costosas y menos convenientes.

La implementación de soluciones tecnológicas como un sistema de gestión de menús digitalizado podría transformar radicalmente estos aspectos, mejorando no solo la gestión interna sino también la experiencia del usuario, alineando a la Universidad de Las Tunas con las prácticas contemporáneas en la gestión de servicios alimentarios y reforzando su compromiso con la innovación y la mejora continua.

A partir de las insuficiencias detectadas se define como **Problema de la Investigación:**

Las insuficiencias en la gestión del menú limitan la calidad de los servicios en los comedores de la Universidad de Las Tunas.

Se define como **objeto de estudio de la investigación:** La gestión del menú.

Se precisa como **campo de acción:** La gestión de información del menú en los comedores de la Universidad de Las Tunas.

Se persigue como **objetivo de la investigación:** Desarrollo de un sistema informático para la gestión del menú en los comedores de la Universidad de Las Tunas.

Se defiende la **idea** de que, con el desarrollo de un sistema informático para la gestión de menú en los comedores de la Universidad de Las Tunas se contribuirá a aumentar la calidad de los servicios.

Tareas de la investigación:

1. Diagnosticar la gestión del menú en los comedores de la Universidad de Las Tunas.
2. Analizar los sistemas para la gestión de menú de los comedores.
3. Seleccionar la metodología, tecnologías, lenguajes y herramientas para el desarrollo del software.
4. Implementar un sistema para la gestión de menú en los comedores de la Universidad de Las Tunas.
5. Validar el sistema informático implementado.

Para cumplir con las tareas propuestas anteriormente se emplearon métodos científicos para la investigación, tanto teóricos como empíricos.

Como métodos teóricos se emplearon:

El histórico-lógico: Se emplea para estudiar el surgimiento y evolución de la gestión de la información del menú en los comedores de la Universidad de Las Tunas y sentar las bases e ideas del trabajo a realizar.

El analítico-sintético: Se utiliza en el análisis del proceso de gestión de la información del menú en los comedores de la Universidad de Las Tunas, lo que posibilitó establecer las normas que regulan el comportamiento del mismo.

El método de la modelación: facilita representar las abstracciones del proceso de gestión de la información del menú de alimentación y la reserva de tickets en los comedores de la Universidad de Las Tunas.

Como métodos empíricos se emplearon:

La entrevista: Se emplea para recibir retroalimentación por parte de los trabajadores que intervienen en el proceso de gestión de la información del menú en los comedores de la Universidad de Las Tunas y comprender así la forma en que se llevaba a cabo este proceso con vista a realizar el levantamiento de requisitos.

La observación: Se utiliza para analizar el trabajo dentro del proceso de gestión de la información del menú en los comedores de la Universidad de Las Tunas y detectar las insuficiencias de la problemática.

El presente trabajo de diploma se encuentra estructurado en tres capítulos:

Capítulo I. Fundamentación teórica: en este capítulo se describe la problemática que surge a raíz de la gestión de la información del menú en los comedores de la Universidad de Las Tunas. Se estudia la existencia de sistemas que gestionen los procesos de gestión de la información de menús de alimentación en los comedores y se definen las tecnologías y herramientas a utilizar para la realización de la aplicación.

Capítulo II. Análisis y diseño del sistema: se realiza la modelación del sistema utilizando la metodología ágil para el desarrollo de software Scrum para la Arquitectura de Software como primer paso en el desarrollo de la herramienta y brindando una perspectiva general de los requerimientos necesarios para dar cumplimiento al desarrollo de la aplicación.

Capítulo III. Implementación del sistema: este capítulo aborda la descripción del sistema que se creará para la gestión de la información del menú en los comedores de la Universidad de Las Tunas. Se explica la interacción con el usuario, la protección y seguridad del sistema, el tratamiento de errores y los beneficios que se obtendrán con la implementación del sistema.

CAPÍTULO I. MARCO TEÓRICO REFERENCIAL

Este capítulo está dirigido a la gestión de la información del menú de alimentación en los comedores de la Universidad de Las Tunas y los sistemas de gestión de los procesos de alimentación como paso para comprender la importancia de estos. Se conceptualizará el proceso que será informatizado, conforme a las tendencias de desarrollo de software actuales, las tecnologías y lenguajes de modelación que se utilizarán en el análisis y diseño del sistema. Además, se abordarán los conceptos fundamentales, la metodología y herramientas utilizadas en el desarrollo del sistema.

1.1 Gestión de la alimentación

La gestión de la alimentación es una disciplina crucial en la administración de empresas [4] . Se ocupa de la planificación, organización, motivación y control de los recursos necesarios para garantizar una alimentación adecuada y saludable. Implica el uso de metodologías, herramientas y técnicas específicas para definir estrategias, asignar recursos, establecer horarios y supervisar la ejecución de acciones relacionadas con la alimentación. A diferencia de otras formas de gestión, la gestión de la alimentación se enfoca en un ciclo continuo. Tiene objetivos específicos, como mejorar la salud y el bienestar de las personas a través de una alimentación balanceada. Además, considera restricciones como el tiempo, el costo, la calidad y el alcance de los alimentos disponibles. Requiere coordinación entre diversos elementos, como la selección de ingredientes, la preparación de comidas y la distribución eficiente. La gestión de la alimentación beneficia tanto a las organizaciones como a los individuos. Contribuye al cumplimiento de objetivos estratégicos y operativos al alinear las prácticas alimentarias con los valores y la visión de la organización. Además, mejora la satisfacción de los clientes al proporcionar opciones saludables y deliciosas. Al optimizar los recursos y minimizar el desperdicio, también promueve la eficiencia y la sostenibilidad.

1.1.1 Gestión del menú de alimentación en las universidades

La gestión de la alimentación en las universidades aborda la planificación, organización y control de los recursos relacionados con la alimentación en entornos académicos [5]. A diferencia de la gestión de alimentación tradicional, en las universidades se centra en un ciclo continuo de actividades, que incluyen la selección de alimentos, la preparación de comidas, la distribución eficiente y la atención a las necesidades nutricionales de los estudiantes y el personal universitario.

La gestión del menú en los comedores universitarios implica un proceso sistemático que incluye la selección de ingredientes, la creación de opciones equilibradas y la evaluación constante de la satisfacción de los usuarios. Este proceso no solo busca ofrecer comidas variadas y atractivas, sino también garantizar que se cumplan los requerimientos nutricionales necesarios para apoyar el rendimiento académico y el bienestar general. Además, la gestión del menú debe ser flexible para adaptarse a cambios en la demanda, preferencias alimentarias y disponibilidad de recursos.

La gestión de la alimentación en las universidades presenta características únicas. Por un lado, existe una alta incertidumbre debido a las variaciones en la demanda, los gustos y las preferencias de los comensales. Además, la interdependencia entre los diferentes componentes del sistema alimentario (como la cocina, el comedor y la logística) requiere una coordinación eficiente.

El impacto de una buena gestión de la alimentación, incluyendo la planificación del menú, se refleja en la salud y el bienestar de la comunidad universitaria, así como en la sostenibilidad y la calidad de vida en el campus. La gestión de la alimentación en las universidades busca proporcionar opciones saludables y equilibradas, optimizar los recursos disponibles y garantizar una experiencia alimentaria positiva. Al aplicar buenas prácticas, involucrar a los estudiantes y el personal en la toma de decisiones y promover la colaboración, se contribuye al éxito académico y al bienestar general en el entorno universitario.

1.1.2 Proceso de gestión de la alimentación en la Universidad de Las Tunas.

La Universidad de Las Tunas “Vladimir Ilich Lenin” es constituida el 25 de enero de 2010, adscrita al Ministerio de Educación Superior, mediante el acuerdo No. 6767, adoptado por el Comité Ejecutivo del Consejo de Ministros. Esta cuenta con una matrícula de 1064 estudiantes del curso diurno. El claustro está compuesto por un total de 875 docentes. Contamos además con 414 trabajadores no docentes [1] .

Con el desarrollo de las nuevas tecnologías y la incorporación de estas a la mayoría de los procesos de la sociedad, la Universidad no ha estado exenta de las problemáticas y desventajas que trae la no implementación de estas en la vida universitaria. La gestión de la alimentación en la Universidad de Las Tunas viene afrontando varias deficiencias derivadas de la falta de las herramientas necesarias para incorporar las bondades de las nuevas tecnologías.

La gestión de la alimentación en la Universidad de Las Tunas funciona de la siguiente manera:

El proceso inicia por parte del Almacenero del Campus Pepito Tey, solicitando a su homólogo en el Campus Vladimir Ilich Lenin la existencia de productos en su almacén. Luego este presenta un informe detallado sobre el inventario disponible en ambos almacenes al Departamento de Alimentos. Este documento, que refleja la existencia de productos almacenados, sirve como base para la propuesta de menús que se desarrolla en una sesión de coordinación interdepartamental, donde participan de manera conjunta la jefa del Departamento de Alimentos y el técnico, el director de Logística y el especialista, la FEU, la UJC y en ocasiones participan miembros del PCC y el Sindicato. Una vez consensuado el menú en dicha reunión, el documento es enviado al Departamento de Contabilidad donde son aprobados los menús y se devuelven al Almacenero para su validación técnica, que incluye la asignación de códigos específicos a cada ítem del menú. Posteriormente, este documento codificado regresa al Departamento de Contabilidad, instancia responsable de realizar la asignación de precios a cada uno de los platos, considerando criterios de costos, márgenes de utilidad y políticas institucionales.

Finalizado este paso, el Técnico de Alimentos recibe el menú ya estructurado, con códigos y precios definidos, y procede a su distribución oficial hacia el comedor universitario, espacio donde se implementará la venta de tickets correspondientes. Cabe destacar que, atendiendo a las necesidades y normativas institucionales, se diseñan y gestionan dos menús diferenciados: uno destinado a los estudiantes y otro al cuerpo docente, garantizando así la adecuación a los perfiles y requerimientos de cada grupo.

Conociendo los precios de cada menú, estos son enviados a la persona encargada de la venta de los tickets, la cual se realiza con un día de antelación en efectivo. Al final de la venta, se elabora un documento controlando los platos que fueron vendidos.

En el caso de los viernes, se procede a la venta de los almuerzos del sábado y lunes siguiente.

A partir de este análisis se detectaron varias deficiencias en el proceso:

1. No existe un mecanismo centralizado de control de inventario.
2. En el proceso actual intervienen muchos trabajadores, provocando que el flujo sea más complejo.
3. Para reservar el ticket hay que estar presente de forma obligatoria en la Universidad.
4. No es posible hacer reservas o pagos en línea.

1.2 Soluciones de software en la gestión del menú de alimentación

Un software para la gestión del menú de alimentación permite simplificar todos los procesos relacionados con la planificación, preparación y distribución de alimentos. Este tipo de software, conocido como software de gestión de menús, es una solución especializada que puede estar disponible en la nube (cloud) o alojada en servidores locales, diseñada específicamente para cubrir las necesidades de instituciones y organizaciones en el ámbito de la alimentación [6].

Un software de gestión de menús es una herramienta integral que centraliza y optimiza las actividades relacionadas con la alimentación. Desde un solo programa, es posible gestionar la planificación de menús equilibrados y nutritivos, la selección de materias primas, el control de inventarios y la distribución eficiente de comidas. Además, facilita la coordinación entre nutricionistas, chefs y administradores para garantizar que se cumplan los estándares de calidad y las expectativas de los comensales.

Este tipo de software no solo agiliza los procesos operativos, sino que también contribuye a mejorar la experiencia alimentaria. Al ofrecer opciones personalizadas y adaptadas a las necesidades nutricionales, se promueve un entorno más saludable y sostenible. Además, permite la recopilación de datos sobre preferencias y hábitos alimentarios, lo que facilita la toma de decisiones informadas para futuras planificaciones de menús.

Softwares similares internacionales:

gsBase alimentación

Solución de gestión ERP de alimentación personalizable en función de las necesidades de cada empresa, que abarca tanto el área de producción y fabricación como el de distribución y venta. Como software de distribución y producción alimentaria se centra en la distribución al por mayor de alimentos y el control de la producción [6].

SAP Business One Alimentación

Software ERP de alimentación que facilita la digitalización de la industria alimentaria. Un sistema de gestión inteligente que integra tecnologías móviles, internet de las cosas, aprendizaje automático y que ofrece una visibilidad 360° de todas las áreas y procesos del negocio de la producción y distribución alimentaria [6].

Softwares similares nacionales:

Versat Sarasola

Sistema de gestión contable-financiero que representa un ejemplo de sustitución de importaciones en materia de aplicaciones informáticas. Se reconoce como el primer sistema cubano de gestión autóctono y más versátil. Este software es hoy un producto maduro, altamente configurable y de resultados comprobados, tanto en el sector empresarial como presupuestado [7].

CONTACC

Sistema de control de acceso a comedores en la UCI. Este sistema tiene como objetivo fundamental gestionar los accesos a comedores, ello consiste en: registrar accesos, comprobando que dicha persona existe, tiene autorización para pasar por esa puerta y no ha accedido con anterioridad, sincronizar accesos o actualizar cada cierto tiempo la información registrada en las bases de datos locales con las bases de datos centrales a través del Servicio Web; y brindar un servicio de reportes donde se muestre al usuario la cantidad de personas (por tipo) que han tenido acceso hasta el momento [8].

ESazúcar. Sistema de Comedor Empresa Azucarera Colombia

Sistema para la gestión de la información del menú de alimentación en el Comedor Industria en la Empresa de Servicios a la Agroindustria Azucarera Colombia. Este software tiene como objetivo gestionar los inventarios de almacén, menús de alimentos, ventas y finanzas de la Empresa [8].

A partir del análisis exhaustivo de las aplicaciones existentes y sus funcionalidades, se ha determinado que estas no satisfacen los requerimientos específicos de la Universidad de Las Tunas, pues no cumplen todos los requerimientos para darle solución a la problemática que implica este proceso. Estos sistemas requieren pagos de suscripción, están adaptados a los procesos de la empresa que los desarrolla, o se encuentran limitados en alcance y funcionalidad, sin embargo han servido como

una fuente valiosa de inspiración. Las características más destacadas y las funcionalidades más eficientes han sido cuidadosamente seleccionadas y adaptadas para el diseño de una herramienta a medida. Esta herramienta personalizada promete no solo abordar las necesidades únicas de la Universidad de Las Tunas, sino también incorporar las mejores prácticas de la industria, asegurando así una solución robusta y a la vanguardia de la gestión de la alimentación.

1.3 Metodologías, tecnologías, lenguajes y herramientas

En el ámbito del desarrollo de software, la elección de metodologías, tecnologías, lenguajes de programación y herramientas es crucial para el éxito de cualquier proyecto [9]. Estos componentes son los pilares sobre los que se construye una aplicación, determinando su capacidad para adaptarse a los cambios, su facilidad de mantenimiento y su eficacia en resolver problemas complejos. La sinergia entre estos elementos permite a los equipos de desarrollo crear soluciones que no solo cumplen con los requisitos actuales, sino que también están preparadas para las demandas futuras. Así, cada decisión tomada en este proceso es un reflejo de la visión y la estrategia a largo plazo, asegurando que el producto final sea de la más alta calidad y relevancia en el mercado .

La abundancia de tecnologías y herramientas para el desarrollo de sistemas informáticos representa un vasto mar de posibilidades, donde la selección adecuada es fundamental para el éxito del proyecto [10]. Esta elección debe ser estratégica, buscando un equilibrio entre el costo, la flexibilidad y la versatilidad, sin dejar de lado la importancia de alinearse con las tendencias actuales. Al considerar estos aspectos, se garantiza que el desarrollo no solo sea eficiente en términos de recursos, sino también resiliente y capaz de adaptarse a las necesidades cambiantes del mercado y de los usuarios. En este contexto, cada herramienta y tecnología elegida es un ladrillo que contribuye a la construcción de una solución sólida y sostenible.

1.3.1 Metodologías

Las metodologías de desarrollo de software constituyen el esqueleto metodológico que guía a los equipos a través del complejo proceso de creación de soluciones informáticas. Estas metodologías no son meras técnicas; son filosofías integrales que encapsulan prácticas, herramientas y una mentalidad que busca maximizar la eficiencia y la calidad del software producido. El propósito fundamental de estas metodologías es estructurar y dinamizar los equipos de desarrollo para que puedan abordar las tareas de programación de manera óptima y sistemática [10].

En el espectro actual, las metodologías se bifurcan en dos corrientes principales: las ágiles y las tradicionales. Las metodologías ágiles, como Scrum y Kanban, se centran en la flexibilidad, la entrega continua y la capacidad de adaptación a cambios imprevistos, promoviendo la colaboración y la comunicación constante. Por otro lado, las metodologías tradicionales, como el modelo en cascada, enfatizan una planificación rigurosa y una secuencia lineal de fases, ofreciendo una estructura más predecible y documentada. Ambas corrientes tienen sus méritos y se adaptan a diferentes tipos de proyectos y entornos de trabajo, reflejando la diversidad y la riqueza del campo del desarrollo de software.

Metodologías de desarrollo de software tradicionales: Las metodologías de desarrollo de software tradicionales se caracterizan por definir total y rígidamente los requisitos al inicio de los proyectos de ingeniería de software. Los ciclos de desarrollo son poco flexibles y no permiten realizar cambios, al contrario que las metodologías ágiles; lo que ha propiciado el incremento en el uso de las segundas.

La organización del trabajo de las metodologías tradicionales es lineal, es decir, las etapas se suceden una tras otra y no se puede empezar la siguiente sin terminar la anterior. Tampoco se puede volver hacia atrás una vez se ha cambiado de etapa. Estas metodologías, no se adaptan nada bien a los cambios, y el mundo actual cambia constantemente.

Metodologías de desarrollo de software ágiles: Las metodologías ágiles de desarrollo de software son las más utilizadas hoy en día debido a su alta flexibilidad y agilidad. Los equipos de trabajo que las utilizan son mucho más productivos y eficientes, ya que saben lo que tienen que hacer en cada momento. Además, la metodología permite adaptar el software a las necesidades que van surgiendo por el camino, lo que facilita construir aplicaciones más funcionales.

Las metodologías ágiles se basan en la metodología incremental, en la que en cada ciclo de desarrollo se van agregando nuevas funcionalidades a la aplicación final. Sin embargo, los ciclos son mucho más cortos y rápidos, por lo que se van agregando pequeñas funcionalidades en lugar de grandes cambios [10].

Este tipo de metodologías permite construir equipos de trabajo autosuficientes e independientes que se reúnen cada poco tiempo para poner en común las novedades. Poco a poco, se va construyendo y puliendo el producto final, a la vez que el cliente puede ir aportando nuevos requerimientos o correcciones, ya que puede comprobar cómo avanza el proyecto en tiempo real.

Tras un análisis meticuloso de las distintas metodologías de desarrollo de software, se ha tomado la decisión estratégica de adoptar el siguiente conjunto específico de metodologías para el desarrollo del proyecto. Estas metodologías han sido seleccionadas por su alineación con los objetivos del proyecto, su capacidad para facilitar la colaboración eficiente y su compatibilidad con las prácticas de desarrollo ágil. La combinación de estas metodologías proporcionará un marco de trabajo flexible y adaptable, capaz de responder a los desafíos dinámicos del desarrollo de software y de satisfacer las expectativas de entrega y calidad.

Scrum: Es un marco de gestión de proyectos de metodología ágil que ayuda a los equipos a estructurar y gestionar el trabajo mediante un conjunto de valores, principios y prácticas. Scrum se inspira en el deporte del rugby, donde los jugadores trabajan juntos para avanzar hacia un objetivo común. Se aplica principalmente al

desarrollo de software, pero también se puede utilizar en otros tipos de proyectos complejos y creativos [11].

El concepto de Scrum se basa en la idea de que el desarrollo de software es un proceso empírico, que requiere adaptación, aprendizaje e inspección continua. Propone un enfoque iterativo e incremental, que divide el proyecto en ciclos cortos y fijos llamados sprints, que suelen durar entre una y cuatro semanas. Cada sprint tiene como resultado un producto funcional y potencialmente entregable, que se puede probar y mejorar en los siguientes sprints. También promueve la colaboración y la comunicación entre los miembros del equipo y con el cliente, para asegurar que se satisfagan las necesidades y expectativas de los usuarios finales.

Algunas de las principales características que distinguen a Scrum de otros marcos de gestión de proyectos son:

Los roles: Scrum define tres roles esenciales para el éxito del proyecto: el propietario del producto, el facilitador y el equipo Scrum. El propietario del producto es el responsable de definir y priorizar el trabajo que el equipo debe realizar, así como de validar el resultado final. El facilitador es el responsable de facilitar y guiar el proceso Scrum, asegurando que se sigan las reglas y se resuelvan los obstáculos. El equipo Scrum es el responsable de diseñar, desarrollar y entregar el producto, de forma autónoma y autoorganizada.

Los artefactos: Scrum utiliza tres artefactos principales para gestionar el trabajo y el progreso del proyecto: el trabajo pendiente del producto, el trabajo pendiente del sprint y el incremento. El trabajo pendiente del producto es una lista ordenada de las características, funciones y requisitos que el producto debe tener. El trabajo pendiente del sprint es una lista de las tareas que el equipo se compromete a realizar en el sprint actual. El incremento es el producto funcional y potencialmente entregable que se obtiene al final de cada sprint [5].

Los eventos: Scrum establece cinco eventos principales para planificar, ejecutar, revisar y mejorar el proyecto: la planificación del sprint, el Scrum diario, la revisión del

sprint, la retrospectiva del sprint y el refinamiento del trabajo pendiente del producto. La planificación del sprint es una reunión donde el equipo selecciona y desglosa el trabajo que realizará en el sprint. El Scrum diario es una reunión breve donde el equipo comparte el estado y los planes del día. La revisión del sprint es una reunión donde el equipo presenta el incremento al propietario del producto y a las partes interesadas, y recibe su retroalimentación. La retrospectiva del sprint es una reunión donde el equipo reflexiona sobre el sprint y propone acciones de mejora. El refinamiento del trabajo pendiente del producto es un proceso continuo donde el equipo revisa y actualiza el trabajo pendiente del producto.

Scrum ofrece una serie de ventajas para los equipos y los clientes, tales como:

- Mayor adaptabilidad y flexibilidad, al permitir responder a los cambios y a las necesidades del cliente de forma rápida y eficaz.
- Mayor calidad y funcionalidad del producto, al permitir probar, verificar y mejorar el producto de forma continua e incremental.
- Mayor velocidad y productividad, al permitir entregar valor al cliente de forma frecuente y regular.
- Mayor colaboración y comunicación, al permitir involucrar a los miembros del equipo y al cliente en el proceso de desarrollo y en la toma de decisiones.
- Mayor motivación y satisfacción, al permitir a los miembros del equipo asumir la responsabilidad y el control de su trabajo, y al permitir al cliente obtener el producto que desea y necesita.

1.3.2 Frameworks

Un framework de desarrollo es un conjunto de herramientas y código que facilita la creación de aplicaciones [12]. Proporciona una estructura, una funcionalidad y una seguridad comunes para el desarrollo, lo que permite a los desarrolladores ahorrar tiempo y esfuerzo, y evitar errores frecuentes. Entre los frameworks de desarrollo más robustos se encuentran Spring-Boot, ASP.NET, Angular, Flutter, React, Lavarel

y Django. Los frameworks de desarrollo facilitan la estructura y la organización del código, lo que mejora la legibilidad, la mantenibilidad y la reutilización del mismo. Proporcionan una funcionalidad y una seguridad comunes, lo que evita tener que reinventar la rueda y protege las aplicaciones web de ataques y vulnerabilidades. Reducen el tiempo y el esfuerzo de desarrollo, lo que permite entregar las aplicaciones web más rápido y con menos coste. Mejoran la calidad y la funcionalidad de las aplicaciones web, lo que aumenta la satisfacción y la fidelización de los usuarios. Fomentan la colaboración y la comunicación entre los desarrolladores, lo que facilita el trabajo en equipo y la resolución de problemas.

La selección de los frameworks de desarrollo es un paso crítico que influye directamente en la eficiencia y sostenibilidad del proyecto. Esta decisión estratégica no solo garantiza una curva de aprendizaje más suave para el equipo de desarrollo, sino que también asegura una mayor coherencia, compatibilidad, se facilita la integración y se acelera el proceso de desarrollo, permitiendo al equipo centrarse en la innovación y la calidad del producto final.

Spring Boot: es un framework de desarrollo web que facilita la creación de aplicaciones web basadas en Spring, que se pueden ejecutar de forma independiente y sin necesidad de desplegar archivos WAR. Spring Boot ofrece una visión opinada de la plataforma Spring y de las bibliotecas de terceros, para que pueda empezar con el mínimo esfuerzo. La mayoría de las aplicaciones Spring Boot requieren poca o ninguna configuración de Spring [13].

El concepto de Spring Boot se basa en la idea de que el desarrollo de software es un proceso empírico, que requiere adaptación, aprendizaje e inspección continua. Propone un enfoque convencional sobre configuracional, que asume y añade automáticamente las configuraciones y dependencias más comunes para el desarrollo web, evitando tener que definir las manualmente. También proporciona características listas para producción, como métricas, chequeos de salud y configuración externa.

Algunas de las principales características que distinguen a Spring Boot de otros frameworks de desarrollo web son:

Los iniciadores: son dependencias que agrupan otras dependencias comunes para una funcionalidad específica, como el acceso a datos, la seguridad o la web. Estos simplifican el manejo de las dependencias y evitan los conflictos de versiones.

La autoconfiguración: es un mecanismo que detecta y configura automáticamente los beans y las propiedades de Spring necesarios para el funcionamiento de la aplicación, según las dependencias y el entorno disponibles. La autoconfiguración se puede personalizar o anular mediante anotaciones o archivos de propiedades.

El embebido: es una característica que permite embeber un servidor web, como Tomcat, Jetty o Undertow, dentro de la aplicación, sin necesidad de desplegar archivos WAR. El embebido facilita el desarrollo, el despliegue y la ejecución de la aplicación.

Los actuators: son endpoints que exponen información y operaciones sobre el estado y el funcionamiento de la aplicación, como las métricas, los chequeos de salud, los registros o los beans de Spring. Los actuators se pueden acceder mediante HTTP o JMX, y se pueden personalizar o securizar.

Spring Boot ofrece una serie de ventajas para los desarrolladores y los clientes:

- Mayor rapidez y facilidad para desarrollar aplicaciones web, al reducir la configuración y el código necesarios.
- Mayor calidad y funcionalidad de las aplicaciones web, al utilizar las mejores prácticas y las características de Spring.
- Mayor compatibilidad y flexibilidad, al integrarse con otras tecnologías y bibliotecas de Spring y de terceros.
- Mayor productividad y satisfacción, al centrarse en las características de negocio y no en la infraestructura.

Flutter: es un framework de desarrollo multiplataforma de código abierto creado por Google, que facilita la creación de aplicaciones de escritorio, web, móviles nativas para iOS y Android desde una única base de código. Flutter ofrece una experiencia de desarrollo rápida y flexible con su motor de renderizado propio y su lenguaje de programación Dart, lo que permite a los desarrolladores construir interfaces de usuario hermosas y compiladas de forma nativa. La mayoría de las aplicaciones Flutter requieren poca o ninguna configuración específica de plataforma [14].

El concepto de Flutter se basa en la idea de que el desarrollo de softwares debe ser una experiencia unificada y no dependiente de la plataforma. Propone un enfoque de diseño primero, donde los widgets y las animaciones juegan un papel central, y añade automáticamente las configuraciones y dependencias más comunes para el desarrollo multiplataforma, evitando tener que definir las manualmente. También proporciona características listas para producción, como compilación en caliente, perfiles de rendimiento y herramientas de inspección.

Algunas de las principales características que distinguen a Flutter de otros frameworks de desarrollo multiplataforma son [15]:

Los widgets: son los bloques de construcción de la interfaz de usuario en Flutter, que representan elementos inmutables y autodescriptivos. Estos simplifican la creación de interfaces complejas y mantienen la consistencia en diferentes plataformas.

La compilación en caliente: es un mecanismo que permite a los desarrolladores ver los cambios en el código de manera instantánea en la aplicación en ejecución, sin necesidad de reiniciarla. La compilación en caliente aumenta la productividad y facilita la iteración rápida.

El motor de renderizado: es una característica que permite a Flutter dibujar widgets directamente en el canvas, lo que proporciona un alto rendimiento y una experiencia de usuario fluida. El motor de renderizado es personalizable y extensible.

Las herramientas de desarrollo: son características que ayudan a los desarrolladores a diagnosticar y corregir problemas de rendimiento y diseño, como el Inspector de

widgets y el rastreador de rendimiento. Las herramientas de desarrollo son accesibles desde la línea de comandos o desde IDEs compatibles.

Flutter ofrece una serie de ventajas para los desarrolladores y los clientes:

- Mayor rapidez y facilidad para desarrollar aplicaciones multiplataforma, al reducir la necesidad de escribir código específico de plataforma.
- Mayor calidad y coherencia de los softwares, al proporcionar una amplia gama de widgets y herramientas de diseño.
- Mayor compatibilidad y flexibilidad, al permitir la integración con servicios y bibliotecas nativas de iOS y Android.
- Mayor productividad y satisfacción, al centrarse en la experiencia del usuario y la estética de la aplicación.

Desarrollo de aplicaciones multiplataforma

El desarrollo de aplicaciones multiplataforma permite a los desarrolladores utilizar un lenguaje de programación y una base de código para crear una aplicación para múltiples plataformas. El desarrollo de aplicaciones de este tipo es menos costoso y requiere menos tiempo que el desarrollo de aplicaciones nativas. Este proceso también permite a los desarrolladores crear una experiencia más coherente para los usuarios en todas las plataformas. Este enfoque puede tener inconvenientes en comparación con el desarrollo de aplicaciones nativas, como el acceso limitado a la funcionalidad de los dispositivos nativos. Sin embargo, Flutter tiene características que hacen que el desarrollo de aplicaciones multiplataforma sea más fluido y de alto rendimiento [8] .

1.3.3 Lenguajes de programación

En la elección de los frameworks y lenguajes para el desarrollo, se ha priorizado la sinergia entre Spring Boot y Flutter, aprovechando la disponibilidad y familiaridad de los lenguajes de programación Java y Dart respectivamente. Java, emparejado con Spring Boot, es reconocido por su robustez y un ecosistema extenso que facilita el

desarrollo de aplicaciones backend escalables. Por otro lado, Dart, como el lenguaje detrás de Flutter, destaca por su simplicidad y eficiencia, lo que lo hace ideal para la creación de interfaces de usuario ricas y fluidas. La combinación de estas tecnologías proporciona una plataforma cohesiva y potente, capaz de entregar aplicaciones con un rendimiento superior y una experiencia de usuario de alta calidad.

Java: Es un lenguaje de programación y una plataforma informática que permite desarrollar y ejecutar aplicaciones y servicios en diversos dispositivos y sistemas operativos. Se caracteriza por ser un lenguaje orientado a objetos, compilado, interpretado, concurrente, seguro y portable [16].

Algunas de las principales características que distinguen a Java de otros lenguajes de programación son:

La orientación a objetos: Java se basa en el paradigma de la programación orientada a objetos, que consiste en modelar el mundo real mediante clases, objetos, atributos y métodos, que se relacionan entre sí mediante los conceptos de herencia, polimorfismo, abstracción y encapsulamiento.

La compilación e interpretación: Java combina la compilación y la interpretación, lo que significa que el código fuente se compila en un código intermedio llamado bytecode, que luego se interpreta y ejecuta por la JVM . Esto permite que el código Java sea portable, eficiente y seguro.

La concurrencia: Java permite el uso de múltiples hilos de ejecución que pueden realizar tareas simultáneas o paralelas, lo que mejora el rendimiento y la funcionalidad de las aplicaciones y proporciona mecanismos para crear, sincronizar y gestionar los hilos de forma sencilla y segura.

La seguridad: Java ofrece un alto nivel de seguridad, tanto a nivel de código como de datos, lo que evita ataques, virus, intrusiones o manipulaciones maliciosas. Utiliza un sistema de seguridad basado en permisos, firmas digitales, cifrado y sandbox, que protege la integridad y la privacidad de las aplicaciones.

La portabilidad: Java es un lenguaje portable, lo que significa que el mismo código Java se puede ejecutar en cualquier máquina que tenga instalada la JVM, sin importar el sistema operativo, el procesador o la arquitectura. Esto facilita la distribución y el despliegue de las aplicaciones Java.

Ventajas de Java:

Mayor productividad y facilidad para desarrollar aplicaciones, al utilizar un lenguaje simple, claro, estructurado y modular.

- Mayor calidad y funcionalidad de las aplicaciones, al utilizar un lenguaje robusto, estable, escalable y compatible.
- Mayor compatibilidad y flexibilidad, al integrarse con otras tecnologías y bibliotecas de Java y de terceros.
- Mayor alcance y versatilidad, al permitir desarrollar aplicaciones para diversos ámbitos y sectores, como la web, el móvil, el escritorio o el empresarial.

Dart: Es un lenguaje de programación moderno y una plataforma para el desarrollo de aplicaciones web, móviles y de escritorio. Se caracteriza por ser un lenguaje orientado a objetos, compilado en código nativo o JavaScript, y optimizado para interfaces de usuario ricas y fluidas [17].

Algunas de las principales características que distinguen a Dart de otros lenguajes de programación son:

La orientación a objetos: Dart se basa en el paradigma de la programación orientada a objetos, facilitando la creación de estructuras de datos complejas y la reutilización de código mediante clases, objetos, atributos y métodos. Incorpora conceptos modernos como mixins, interfaces y clases abstractas, que enriquecen las posibilidades de herencia, polimorfismo, abstracción y encapsulamiento.

La compilación y transpilación: Dart puede compilarse en código nativo para plataformas móviles y de escritorio, o transpilarse a JavaScript para aplicaciones

web, lo que lo hace versátil y eficiente. La máquina virtual de Dart (Dart VM) permite la ejecución de código con tiempos de arranque rápidos y un rendimiento predecible.

La concurrencia: Aunque Dart no utiliza hilos tradicionales, ofrece "isolates", que son trabajadores que ejecutan código en paralelo sin compartir memoria, evitando así problemas de concurrencia y mejorando la seguridad y el rendimiento.

La seguridad: Dart proporciona un entorno de ejecución seguro, con un fuerte sistema de tipos y análisis estático que ayuda a prevenir errores comunes y mejora la calidad del código. Además, el aislamiento de memoria entre "isolates" protege contra efectos secundarios no deseados y vulnerabilidades.

La portabilidad: El código Dart se puede ejecutar en cualquier plataforma que soporte la Dart VM o en cualquier navegador moderno a través de la transpilación a JavaScript, lo que facilita la creación de aplicaciones multiplataforma.

Ventajas de Dart:

- Mayor productividad y facilidad para desarrollar aplicaciones, gracias a características como la compilación en caliente, que permite ver los cambios en tiempo real sin reiniciar la aplicación.
- Mayor calidad y funcionalidad de las aplicaciones, al proporcionar un amplio conjunto de bibliotecas y herramientas para el desarrollo de interfaces de usuario.
- Mayor compatibilidad y flexibilidad, al integrarse con otras tecnologías y bibliotecas, y al ser el lenguaje principal para el desarrollo con el framework Flutter.
- Mayor alcance y versatilidad, al permitir desarrollar aplicaciones para diversos ámbitos y sectores, incluyendo web, móvil y escritorio, con una única base de código.

1.3.4 Sistemas gestores de Base de Datos

En la infraestructura de cualquier aplicación, la gestión de bases de datos es un componente crítico que asegura la integridad, el rendimiento y la escalabilidad de la información. Reconociendo la robustez y la gestión integral que Spring Boot ofrece para bases de datos relacionales, se ha seleccionado PostgreSQL como el sistema de gestión de bases de datos para el proyecto. Este gestor destaca por su conformidad con los estándares SQL, su extensibilidad y su capacidad para manejar grandes volúmenes de datos con eficiencia. La elección de PostgreSQL, en conjunto con Spring Boot, proporciona una solución de almacenamiento de datos sólida y confiable, que facilitará el desarrollo de funcionalidades complejas y garantizará un mantenimiento sencillo y directo.

PostgreSQL: Es un sistema de gestión de bases de datos relacional orientado a objetos y de código abierto, que permite almacenar, manipular y consultar datos de forma eficiente, segura y escalable. Se caracteriza por ser un sistema compatible con el estándar SQL, que soporta múltiples tipos de datos, funciones, procedimientos, índices, extensiones y lenguajes de programación. Se basa en la idea de que los datos se organizan en tablas, que se agrupan en esquemas, que a su vez se agrupan en bases de datos. Cada tabla tiene una o más columnas, que definen los atributos de los datos, y una o más filas, que contienen los valores de los datos. Cada base de datos tiene un catálogo, que almacena la información sobre las tablas, los esquemas y otros objetos de la base de datos [18].

PostgreSQL permite definir tipos de datos personalizados, herencia de tablas, restricciones de chequeo y reglas de reescritura, que facilitan el modelado y la validación de los datos. Ampliar sus funcionalidades mediante el uso de extensiones, que son módulos que añaden nuevas características, como tipos de datos, funciones, operadores, índices o lenguajes de programación. Crear funciones, procedimientos, disparadores y vistas, que son objetos que encapsulan la lógica de negocio y que se pueden ejecutar dentro de la base de datos, utilizando diversos lenguajes de programación, como SQL, PL/pgSQL, Python, Perl, Java o C23.

Permite el acceso simultáneo y seguro de múltiples usuarios a los datos, utilizando un modelo de control de concurrencia multiversión (MVCC), que evita los bloqueos y garantiza la consistencia y el aislamiento de las transacciones. Permite copiar y distribuir los datos entre varios servidores, utilizando diferentes métodos de replicación, como la replicación lógica, la replicación física o la replicación por particionamiento, que mejoran la disponibilidad, el rendimiento y la tolerancia a fallos de la base de datos.

Ventajas de PostgreSQL:

- Mayor calidad y funcionalidad de los datos, al utilizar un sistema robusto, estable, compatible y flexible.
- Mayor velocidad y eficiencia en el manejo de los datos, al utilizar un sistema optimizado, escalable y concurrente.
- Mayor compatibilidad y portabilidad, al integrarse con otras tecnologías y plataformas de código abierto o propietario.
- Mayor seguridad y confiabilidad, al utilizar un sistema que protege los datos de ataques, pérdidas o corrupción.

1.3.5 Patrones de diseño y arquitectura

Los patrones de diseño y arquitecturas de software son fundamentales para la creación de sistemas escalables y mantenibles. Estos enfoques permiten a los desarrolladores estructurar el código de manera que pueda crecer y adaptarse a nuevas necesidades sin comprometer la integridad del sistema. La escalabilidad se refiere a la capacidad del software para manejar un aumento en la carga de trabajo, como más usuarios o transacciones, mientras que la mantenibilidad implica la facilidad con la que el software puede ser modificado o actualizado [19].

El patrón Modelo-Vista-Controlador (MVC) surge con el objetivo de reducir el esfuerzo de programación, necesario en la implementación de sistemas múltiples y

sincronizados de los mismos datos, a partir de estandarizar el diseño de las aplicaciones [20].

MVC (Modelo-Vista-Controlador) es un patrón en el diseño de software comúnmente utilizado para implementar interfaces de usuario, datos y lógica de control. Enfatiza una separación entre la lógica de negocios y su visualización. Esta "separación de preocupaciones" proporciona una mejor división del trabajo y una mejora de mantenimiento. Algunos otros patrones de diseño se basan en MVC, como MVVM (Modelo-Vista-modelo de vista), MVP (Modelo-Vista-Presentador) y MVW (Modelo-Vista-Whatever).

Las tres partes del patrón de diseño de software MVC se pueden describir de la siguiente manera:

Modelo: Maneja datos y lógica de negocios.

Vista: Se encarga del diseño y presentación.

Controlador: Enruta comandos a los modelos y vistas.

Modelo: El modelo define qué datos debe contener la aplicación. Si el estado de estos datos cambia, el modelo generalmente notificará a la vista (para que la pantalla pueda cambiar según sea necesario) y, a veces, el controlador (si se necesita una lógica diferente para controlar la vista actualizada). Volviendo a nuestra aplicación de lista de compras, el modelo especificará qué datos deben contener los artículos de la lista (artículo, precio, etc.) y qué artículos de la lista ya están presentes.

Vista: La vista define cómo se deben mostrar los datos de la aplicación. En nuestra aplicación de lista de compras, la vista definiría cómo se presenta la lista al usuario y recibiría los datos para mostrar desde el modelo.

Controlador: El controlador contiene una lógica que actualiza el modelo y/o vista en respuesta a las entradas de los usuarios de la aplicación. Entonces, por ejemplo, nuestra lista de compras podría tener formularios de entrada y botones que nos permitan agregar o eliminar artículos. Estas acciones requieren que se actualice el

modelo, por lo que la entrada se envía al controlador, que luego manipula el modelo según corresponda, que luego envía datos actualizados a la vista.

Sin embargo, es posible que también se desee actualizar la vista para mostrar los datos en un formato diferente, por ejemplo, cambiar el orden de los artículos de menor a mayor precio o en orden alfabético. En este caso, el controlador podría manejar esto directamente sin necesidad de actualizar el modelo.

La decisión de adoptar estos patrones y arquitecturas se basa en la búsqueda de un diseño que soporte el crecimiento y la evolución del software a largo plazo. Al centrarse en la escalabilidad y la mantenibilidad, se asegura que el sistema pueda responder a las demandas cambiantes del negocio y del mercado, manteniendo al mismo tiempo una alta calidad y un rendimiento óptimo. Estos enfoques proporcionan un marco sólido para el desarrollo de aplicaciones complejas y de gran envergadura, donde la flexibilidad y la adaptabilidad son clave para el éxito.

1.3.6 Herramientas

Para el desarrollo del software propuesto se han seleccionado dos entornos de desarrollo esenciales: Android Studio e IntelliJ IDEA . Estas herramientas son ampliamente reconocidas en la comunidad de desarrollo por su robustez, funcionalidades avanzadas y soporte especializado para diferentes plataformas. Además, para el diseño, documentación y prueba de las interfaces de programación (APIs) que se desarrollarán en este proyecto, se utilizará Apidog, una herramienta moderna que facilita la gestión colaborativa de APIs, ofreciendo funciones como simulación de respuestas, validación de esquemas y generación automática de documentación. La combinación de estas herramientas permite optimizar el proceso de desarrollo, garantizando calidad, eficiencia y una correcta integración entre los componentes del sistema.

Android Studio: es el entorno de desarrollo integrado (IDE) oficial para el desarrollo de apps Android [21]. Basado en el potente editor de código y las herramientas de

desarrollo de IntelliJ IDEA , Android Studio ofrece aún más funciones que mejoran tu productividad al crear apps Android, como:

- Un sistema de compilación flexible basado en Gradle
- Un emulador rápido y rico en funciones
- Live Edit para actualizar componentes componibles en emuladores y dispositivos físicos en tiempo real
- Plantillas de código e integración de GitHub para ayudarlo a crear funciones de aplicaciones comunes e importar código de muestra
- Amplias herramientas y marcos de prueba
- Herramientas de pelusa para detectar problemas de rendimiento, usabilidad, compatibilidad de versiones y otros
- Compatibilidad con C++ y NDK
- Compatibilidad integrada con Google Cloud Platform , lo que facilita la integración de Google Cloud Messaging y App Engine

IntelliJ IDEA: es un Entorno de Desarrollo Integrado (IDE) para el desarrollo profesional en Java y Kotlin. Está diseñado para maximizar la productividad del desarrollador y prioriza la privacidad y la seguridad. Realiza las tareas rutinarias y repetitivas por ti, ofreciendo autocompletado de código inteligente, análisis de código estático y refactorizaciones. Te permite centrarte en el lado positivo del desarrollo de software, haciéndolo no solo productivo, sino también una experiencia agradable [22].

Una de las mejores ventajas de IntelliJ IDEA es su capacidad de personalización. Puedes configurar prácticamente cualquier cosa: la apariencia del IDE, el diseño de las ventanas y barras de herramientas, el resaltado de código y mucho más. También hay numerosas maneras de ajustar el editor y personalizar su

comportamiento para agilizar la navegación y eliminar elementos adicionales que te distraigan del código.

Apidog: es un conjunto completo de herramientas que conecta todo el ciclo de vida de la API y ayuda a los equipos de I+D a implementar las mejores prácticas para el desarrollo de diseño de API [23].

Apidog sigue el enfoque API-first, un enfoque de desarrollo donde la API se diseña y desarrolla antes que la interfaz de usuario. Este enfoque ofrece varias ventajas, como:

- Apidog permite a los equipos trabajar en paralelo y establecer un contrato entre servicios, lo que les permite trabajar en múltiples API simultáneamente y mejorar la velocidad de desarrollo.
- La automatización se puede lograr mediante el uso de herramientas que importan archivos de definición de API, lo que reduce el tiempo necesario para desarrollar y lanzar una API.
- Garantiza una buena experiencia para el desarrollador, con API bien diseñadas y bien documentadas, lo que facilita a los desarrolladores usar y reutilizar código, incorporar nuevos desarrolladores y reducir la curva de aprendizaje.
- Al resolver la mayoría de los problemas antes de escribir cualquier código, también puede evitar problemas al integrar las API con las aplicaciones.

Conclusiones del capítulo

La investigación exhaustiva del proceso de gestión del menú en los comedores de la Universidad de Las Tunas ha revelado áreas críticas de mejora. Se ha identificado una necesidad imperiosa de una aplicación especializada en la gestión del menú de alimentación, que promete revolucionar la eficiencia del proceso y mejorar significativamente la calidad de los servicios.

Para lograr este objetivo, se implementará una solución innovadora que se apoyará en la metodología ágil de Scrum para la arquitectura de software. La interfaz de usuario será desarrollada con Flutter, aprovechando su capacidad para crear experiencias ricas y fluidas, mientras que el backend robusto será construido con Spring Boot, conocido por su escalabilidad y seguridad. Los lenguajes de programación Java y Dart serán utilizados para su desarrollo, garantizando así un rendimiento óptimo y una integración sin fisuras.

CAPÍTULO 2. DESCRIPCIÓN Y CONSTRUCCIÓN DE LA SOLUCIÓN PROPUESTA.

Introducción:

En este capítulo se aborda todo lo relacionado con la creación de la propuesta de solución al problema existente, basándose en la metodología ágil SCRUM. Se mostrarán las fases de planificación y ejecución definidas en la metodología escogida. Además, se detallan las iteraciones llevadas a cabo durante los Sprints, exponiéndose las tareas generadas por cada historia de usuario y las pruebas realizadas al sistema.

2.1 Fase de Iniciación

La primera fase de la metodología SCRUM es la de Iniciación. En esta fase, se define el alcance general del proyecto y se establecen las bases para su desarrollo. El Product Owner, en colaboración con el cliente, precisa lo que se necesita mediante la redacción de historias de usuario que se añaden al Product Backlog [24].

El equipo de desarrollo descompone las historias de usuario en tareas más pequeñas y estima el esfuerzo requerido para cada una de ellas. Durante esta fase, también se estudian las tecnologías a utilizar y se construyen prototipos iniciales del sistema si es necesario.

Esta fase dura aproximadamente dos semanas y el resultado es una visión general del sistema, un Product Backlog inicial y una estimación del plazo total del proyecto.

Visión general del sistema

Sistema de gestión de menú en los Comedores de la Universidad de Las Tunas:

Módulo de Administración: Incluye funcionalidades para la gestión de usuarios, roles, pasarelas de pago, accesos, servicios de directorio activo y auditorías, generación de reportes y análisis de trazas dentro del sistema.

Módulo de Tickets: Incluye funcionalidades para la gestión de tickets disponibles, reserva y cancelación de los mismos dentro de los horarios establecidos en el sistema.

Módulo de Menú: Incluye funcionalidades para la gestión de menú, consulta de los productos de almacén en existencia, propuestas, creación y seguimiento de los menús, libro de recetas, ventas de tickets, acceso al comedor y reporte de informes.

Interacciones Clave:

Usuarios: Interactúan con el sistema a través de interfaces de usuario (frontend), donde pueden gestionar sus tickets.

Especialista: Interactúan con el sistema donde pueden crear propuestas de menú y consultar inventarios.

Contadores: Interactúan con el sistema donde pueden aprobar menús y ajustar precios.

Técnicos: Interactúan con el sistema donde pueden efectuar ventas de tickets, controlar accesos al comedor y generar informes.

Administradores: Tienen acceso a funcionalidades adicionales para la administración del sistema, como la revisión de logs y métricas.

Sistema API: Actúa como intermediario entre el frontend y la lógica de negocio, asegurando que todas las operaciones se realicen de manera segura y eficiente.

Sistema Contable de la Universidad: Actúa como proveedor de los datos de los productos en almacén, así como sus códigos y precios.

2.1.2 Product Backlog

El Product Backlog es una lista priorizada de todo el trabajo que debe realizarse en el proyecto. Es un artefacto vivo que evoluciona a medida que se descubren nuevas

necesidades y se refinan las existentes. El Product Owner es responsable de gestionar el Product Backlog, asegurándose de que esté siempre actualizado y refleje las prioridades del negocio. Cada ítem en el Product Backlog, conocido como historia de usuario, debe estar claramente definido y estimado en términos de esfuerzo requerido. Este backlog sirve como la única fuente de requisitos para cualquier cambio que se realice en el producto. Ver Tabla 1.

Tabla 1. Product Backlog

Id	Nombre	Importancia	Estimado	¿Cómo se hace?	Notas
1	Iniciar sesión	150	8	Acceder a la página de login, ingresar usuario/contraseña y autenticar.	Encriptar contraseñas (https).
2	Gestionar usuario	120	5	Crear/editar/eliminar usuarios.	Conectar con API de correo institucional para verificación.
3	Gestionar roles	140	8	Crear/editar/eliminar roles con permisos (ej: acceso a módulos).	Usar RBAC (Role-Based Access Control). Integrar con JWT.
4	Gestionar ventas	150	8	Vender tickets y controlar acceso al comedor.	Generar código QR para el ticket.
5	Consultar productos	100	4	Revisar productos del almacén en existencia para elaborar propuestas de menú.	Debe conectarse con el Sistema Contable de la Universidad.
6	Gestionar	145	9	Elaborar propuestas	

	propuestas			de menú.	
7	Gestionar recetas	100	4	Crear/editar/eliminar recetas.	
8	Gestionar tickets	135	7	Seleccionar comedor, menú, fecha y confirmar reserva.	Limitar 1 reserva x horario para cada usuario.
9	Gestionar notificaciones	100	5	Configurar alertas websocket para eventos (ej: Menú disponible, Propuestas por aprobar).	Usar websocket para notificaciones.
10	Auditar logs	90	6	Almacenar registros de actividad (inicio de sesión, cambios críticos).	Logs inmutables
11	Gestionar logs	70	3	Eliminar logs antiguos según política (ej: 6 meses).	Backup automático antes de eliminar.
12	Generar informes	80	4	Exportar datos a PDF/Excel con filtros (ej: ventas por fecha).	Usar librería jsPDF para PDF y SheetJS para Excel o similar.

2.1.3 Estimación total del proyecto

La estimación total del proyecto es una evaluación priorizada de todo el trabajo que debe realizarse en el proyecto. Es un proceso continuo que evoluciona a medida que se descubren nuevas necesidades y se refinan las existentes. El Product Owner es responsable de gestionar la estimación total del proyecto, asegurándose de que esté siempre actualizada y refleje las prioridades del negocio. Cada ítem en la estimación, conocido como historia de usuario, debe estar claramente definido y estimado en términos de esfuerzo requerido. Esta estimación sirve como la única fuente de requisitos para cualquier cambio que se realice en el producto.

Total de Puntos de Historia: 71 puntos

Velocidad del Equipo: 18 puntos por Sprint.

Duración del Sprint: 2 semanas.

Número de Sprints Necesarios: 4 Sprints

Duración Total del Proyecto: 8 semanas

Por lo tanto, el plazo general del proyecto sería de aproximadamente 8 semanas, asumiendo que la velocidad del equipo es de 18 puntos por Sprint y que no hay cambios significativos en el Product Backlog.

2.1.4 Definición de Roles y Funcionalidades

Antes de definir las historias de usuario, es preciso tener en cuenta la especificación de la audiencia a la que está dirigido el sistema, es decir, los roles y las funciones que realizarán estos usuarios. En Scrum, estos roles se denominan actores del sistema, ya que serán los beneficiados con la aplicación. Los roles identificados son:

Tabla 2. Roles del Sistema

Rol del Sistema	Descripción
Usuario	Interactúa con la aplicación para obtener sus tickets. Es el cliente final de este sistema.
Especialista	Utiliza la aplicación para crear las propuestas de menú y consultar los inventarios.
Contador	Utiliza el sistema para aprobar las propuestas de menú y hacer ajustes de precios.
Técnico	Utiliza el sistema para la venta de tickets, controlar accesos al comedor y generar informes.
Administrador	Interactúa con la aplicación para administrar toda la información, creación de usuarios, roles y asignación de privilegios. Asegura el correcto funcionamiento y seguridad del sistema.

2.1.5 Historias de Usuario

Las historias de usuario (HU) se utilizan para especificar los requisitos del software. En ellas, el Product Owner describe en su propio lenguaje y de manera breve las características que el sistema debe poseer, ya sean requisitos funcionales o no funcionales. El tratamiento de las historias de usuario es muy dinámico y flexible; en cualquier momento pueden eliminarse o reemplazarse por otras más específicas o generales, añadirse nuevas o ser modificadas. Cada historia de usuario es lo suficientemente comprensible y delimitada para que el Equipo de Desarrollo pueda implementarla en un Sprint.

Durante esta fase se identificaron las siguientes Historias de Usuario:

1. Iniciar sesión
2. Gestionar usuario
3. Gestionar roles
4. Gestionar ventas
5. Consultar productos
6. Gestionar propuestas
7. Gestionar recetas
8. Gestionar tickets
9. Gestionar notificaciones
10. Auditar logs
11. Gestionar logs
12. Generar informes

La descripción de cada Historia de Usuario se realiza a través de una plantilla como se muestra en la tabla 3.

Tabla 3. Plantilla de Historia de Usuario

Historia de Usuario			
ID: Identificador único de la historia de usuario.	Título: Un título breve y descriptivo que resuma la historia de usuario.	Prioridad: Nivel de importancia de la historia de usuario (Alta, Media, Baja).	Estimación: Estimación del esfuerzo necesario para completar la historia de usuario, generalmente en puntos de historia o tiempo.
Descripción: Una descripción detallada utilizando el formato “Como [rol del sistema], quiero [funcionalidad] para [beneficio/valor]”.			
Criterios de Aceptación: Lista de condiciones que deben cumplirse para que la historia de usuario se considere completa.			
Notas Adicionales: Cualquier información adicional relevante para la historia de usuario.			

A continuación se describen las Historias de Usuario de mayor prioridad.

Tabla 4. Historia de Usuario 1. Iniciar sesión

Historia de Usuario 1. Iniciar sesión			
ID: 1	Título: Iniciar Sesión	Prioridad: Alta	Estimación: 8 puntos de historia
Descripción: Como usuario, quiero iniciar sesión en la aplicación para acceder a mis datos y funcionalidades personalizadas.			
Criterios de Aceptación: - El formulario de inicio de sesión debe incluir campos para nombre de usuario y contraseña. - El sistema debe validar las credenciales de forma segura. - Los datos deben estar encriptados durante la transmisión. - El usuario debe ser redirigido a su página de inicio después de una autenticación exitosa.			
Notas Adicionales: Asegurarse de que las credenciales se validen de forma segura y que los datos estén encriptados.			

Tabla 5. Historia de Usuario 2. Gestionar usuario

Historia de Usuario 2. Gestionar usuario			
ID: 3	Título: Gestionar usuario	Prioridad: Alta	Estimación: 5 puntos de historia
Descripción: Como nuevo usuario, quiero registrarme en la aplicación para poder crear una cuenta y acceder a sus funcionalidades.			
Criterios de Aceptación: - El formulario de registro debe incluir campos para nombre, correo electrónico y contraseña. - El sistema debe validar que los datos no existan previamente. - Los datos sensibles deben ser encriptados.			
Notas Adicionales: Validar que los datos no existan previamente y encriptar toda la información sensible.			

2.1.6 Restricciones que el sistema debe cumplir

Los requerimientos no funcionales tienen que ver con características que de una u otra forma puedan limitar el sistema, como por ejemplo, el rendimiento, interfaces de usuario, fiabilidad, mantenimiento, seguridad, portabilidad y estándares. Son propiedades o cualidades que el producto debe tener. Debe pensarse en estas propiedades como las características que hacen al producto atractivo, usable, rápido o confiable. Su importancia radica fundamentalmente en el buen funcionamiento del sistema.

Requerimientos de Hardware

Servidor:

- Procesador a 2.5 GHz o superior
- RAM 16GB
- Espacio en disco 40GB mínimo (considerando almacenamiento de logs, backups y datos)
- Conexión de red de alta velocidad

Cliente:

- Procesador a 1.5 GHz o superior
- RAM 4GB (para asegurar un rendimiento adecuado en aplicaciones web modernas)

Requerimientos de Software

Servidor:

- PostgreSQL versión 12 o superior
- Spring Boot 3
- Java versión 17 o superior
- Sistema Operativo Linux (preferiblemente Ubuntu 20.04 LTS o superior)

Cliente:

- Mozilla Firefox en su versión 100 o superior.
- Sistema Operativo Android (12 o superior).

Restricciones en el Diseño y su Implementación:

- El lenguaje a utilizar para su implementación de parte del backend será Java versión 17 (utilizando Spring Boot 3).
- El lenguaje a utilizar para su implementación de parte del frontend será Dart (utilizando Flutter).
- El sistema gestor de BD será PostgreSQL.
- Debe tener en su diseño el logo de la entidad.
- Implementar políticas de seguridad y autenticación utilizando OAuth 2.0 o mecanismos similares.

Requerimientos de Portabilidad:

- La aplicación deberá funcionar en diferentes plataformas siempre que soporten los requisitos de software.

Requerimientos de Seguridad**Integridad:**

- Garantizar la integridad de los datos a través de la autenticación y autorización utilizando OAuth 2.0 o similares.
- La información que se procesa debe tener un respaldo de información para garantizar restauración en caso de fallos. Utilizar PostgreSQL para backups regulares.

Confidencialidad:

- Deberá contar con varios niveles de acceso para permitir el trabajo organizado con el sistema.
- El acceso a la información será permitido solo a los usuarios autorizados. Utilizar políticas de seguridad en Spring Security.

- La información podrá ser modificada solo por personal autorizado. Asegurar que las modificaciones sean auditables y rastreables.

Requerimientos de Apariencia o Interfaz Externa

- La interfaz gráfica debe ser amigable y fácil de emplear para el usuario. Debe ser legible, rápida e interactiva.
- Utilizar frameworks modernos como Flutter para desarrollar una interfaz de usuario responsiva y dinámica.
- Asegurar que la interfaz sea accesible y compatible con múltiples navegadores y dispositivos.
- Implementar pruebas de usabilidad para garantizar una experiencia de usuario óptima.

2.2 Fase de Planificación

Una vez identificadas las historias de usuario del sistema y estimado el esfuerzo necesario para cada una, se procede a la planificación del Sprint. En esta fase, el Product Owner establece la prioridad de cada historia de usuario en el Product Backlog y el Equipo de Desarrollo realiza una estimación del esfuerzo necesario para cada una de ellas utilizando técnicas como Planning Poker.

Durante la Reunión de Planificación del Sprint, que se lleva a cabo al inicio de cada Sprint, el equipo y el Product Owner acuerdan el contenido del Sprint Backlog y determinan los objetivos del Sprint. Esta reunión suele durar unas pocas horas y el resultado es un plan detallado para el Sprint, incluyendo un cronograma y los entregables esperados.

A continuación se detallan los artefactos generados para el primer sprint del proyecto en esta fase de planificación.

Sprint Backlog

Para el Sprint número 1, se seleccionaron las historias de usuario más prioritarias y y se desglosaron en tareas más pequeñas. Dado que la velocidad del equipo es de 18 puntos por Sprint, seleccionaremos historias de usuario que suman hasta 18 puntos. Ver Tabla 6.

Tabla 6. Sprint Backlog

ID	Nombre	Tareas	Estimado
1	Iniciar sesión	- Crear formulario de inicio de sesión - Implementar validación de credenciales - Encriptar datos de transmisión - Redirigir usuario tras autenticación exitosa	8 puntos
2	Gestionar usuario	- Crear usuario - Editar usuario - Eliminar usuario	5 puntos
3	Gestionar notificaciones	- Generar notificaciones - Leer notificaciones - Cambio estado de notificaciones	5 puntos

Definición de Hecho (Definition of Done)

La Definición de Hecho para nuestras historias de usuario incluye:

- Código revisado y aprobado.

- Pruebas unitarias y de integración pasadas.
- Documentación actualizada.
- Despliegue en entorno de prueba.
- Validación de criterios de aceptación.

Objetivo del Sprint (Sprint Goal)

El objetivo del Sprint número 1 es implementar las funcionalidades básicas de autenticación, permitiendo a los usuarios iniciar y cerrar sesión de manera segura.

Tabla 7. Tablero de Tareas (Task Board)

Por hacer	En progreso	Terminadas
Implementar validación de credenciales	Encriptar datos de transmisión	Crear formulario de inicio de sesión
Redirigir usuario tras autenticación exitosa	Editar usuario	Crear usuario
Eliminar usuario		Generar notificaciones
		Leer notificaciones
		Cambio estado de notificaciones

Plan de Liberación (Release Plan)

El plan de liberación describe cómo y cuándo se entregarán las funcionalidades del producto a los usuarios. Para este proyecto, se planeó liberar las funcionalidades básicas de autenticación al final del primer Sprint.

Resumen del Sprint 1:

Historias de Usuario Seleccionadas: Iniciar sesión, Gestionar usuario, Gestionar notificaciones

Total de Puntos de Historia: 18 puntos

Duración del Sprint: 2 semanas

Objetivo del Sprint: Implementar las funcionalidades básicas de autenticación, gestión de usuarios y gestión de notificaciones

2.3 Diseño de la base de datos

El diseño de la base de datos se basa en un modelo relacional centralizado, cuyas características y estructura se detallan en el Diagrama Entidad-Relación (DER) que se presenta a continuación. Está compuesta por 8 tablas normalizadas, cumpliendo con las normas establecidas para el diseño de bases de datos, siendo de estas las más importantes Menu y Recipe. Esta estrategia se adopta para facilitar la consistencia de los datos y la integridad referencial a través de relaciones explícitas (claves primarias y foráneas), permitiendo realizar consultas complejas y transacciones que involucren múltiples aspectos del negocio de forma atómica y eficiente [25].

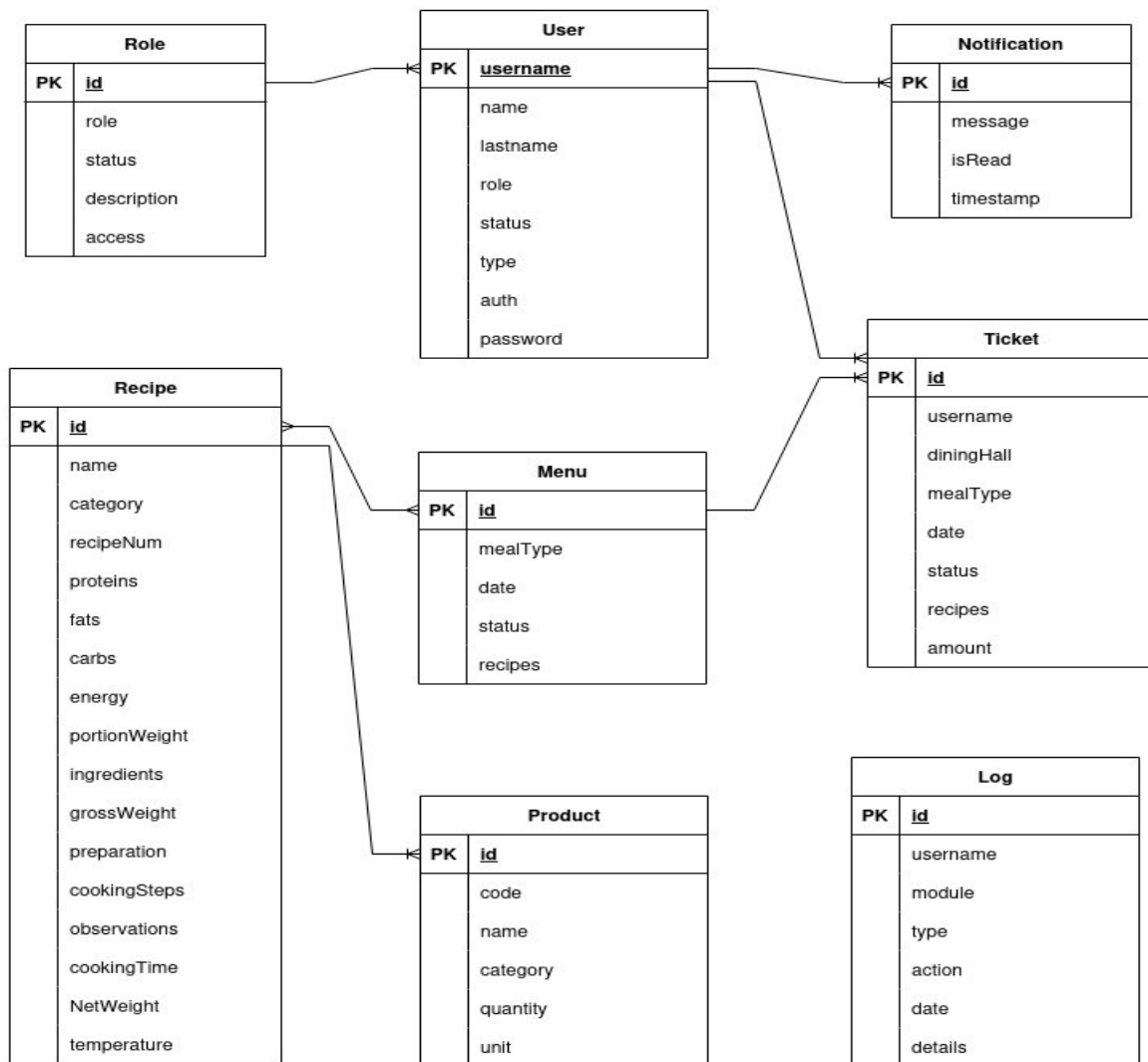


Figura 1. Diseño de la base de datos

2.4 Codificación y Pruebas

La fase de producción requiere de pruebas adicionales y revisiones de rendimiento antes de que el sistema sea trasladado al entorno del cliente. Al mismo tiempo, se deben tomar decisiones sobre la inclusión de nuevas características a la versión actual, debido a cambios durante esta fase. Es posible que disminuya el tiempo que toma cada iteración. Las ideas que han sido propuestas y las sugerencias son documentadas para su posterior implementación.

2.4.1 Codificación

En la implementación del sistema se adopta una arquitectura cliente-servidor, combinando tecnologías robustas para cada capa. Para el desarrollo de la interfaz de usuario y la lógica del lado del cliente, se emplea el framework multiplataforma Flutter junto con el lenguaje de programación Dart, permitiendo la creación de una experiencia de usuario nativa y compilada para diversas plataformas desde una base de código unificada. Complementariamente, para el lado del servidor, se utiliza el framework Spring Boot 3 junto con el lenguaje de programación Java, el cual es ampliamente utilizado para el desarrollo de aplicaciones backend robustas y escalables. En ambas partes de la aplicación, se emplean clases y otros principios de la programación orientada a objetos, ya que, dadas las características del sistema, es esencial estructurar el código de manera modular y mantenible. Además, se ha considerado la reutilización de código a través de componentes reutilizables (como Widgets en Flutter y servicios en Spring Boot), lo que agiliza el proceso de desarrollo y promueve una arquitectura limpia y eficiente en el software. Ver figura 2.

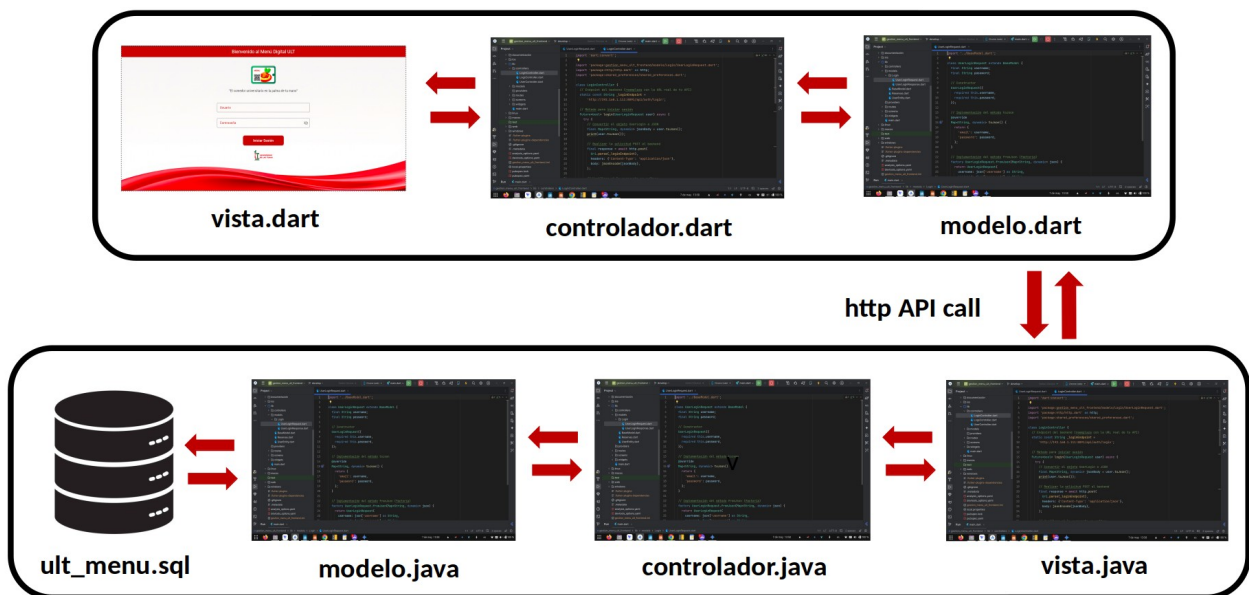


Figura 2. Arquitectura del sistema

2.4.2 Estándar de codificación

Un estándar de codificación comprende todos los aspectos de la generación de código. Si bien los programadores deben implementar un estándar de forma prudente, este debe tender siempre a lo práctico. Un código fuente completo debe reflejar un estilo armonioso, como si un único programador hubiera escrito todo el código de una sola vez. Cuando el proyecto de software incorpore código fuente previo, o bien cuando realice el mantenimiento de un sistema de software creado anteriormente, el estándar de codificación debería establecer cómo operar con la base de código existente. Siguiendo esta conceptualización para la codificación del sistema se tuvieron en cuenta los siguientes estándares:

Declaraciones de variables:

- El nombre que se le da a las variables debe comenzar con la primera letra en minúscula, en caso de que sea un nombre compuesto se empleará la notación Camel Case.

Espacio en blanco:

El uso de espacios en blanco mejora la legibilidad del código agrupando secciones de código que se relacionan lógicamente. Se recomienda usar espacios en blanco entre los operadores lógicos y aritméticos para lograr una mayor legibilidad en el código.

Se usan espacios en blanco en las siguientes circunstancias:

- Aparece un espacio en blanco después de cada coma en las listas de argumentos.
- Todos los operadores binarios son separados de sus operandos con espacios en blanco. Los espacios en blanco no deben separar los operadores unarios, incremento (“++”) y decremento (“- -”) de sus operandos.

Declaraciones de clases

Para la declaración de clases se siguieron las siguientes reglas:

- Ningún espacio en blanco entre el nombre de un método y el paréntesis “(” que abre su lista de parámetros.
- La llave de cierre “)” empieza una nueva línea indentada para ajustarse a su sentencia de apertura correspondiente, excepto cuando no existen sentencias entre ambas, que debe aparecer inmediatamente después de la de apertura “{”.
- Los métodos se separan con una línea en blanco.

Comentarios

- Se recomienda comentar al inicio de cada clase o función y al final de cada bloque de código.
- Se deben escribir comentarios completos y exactos.
- Se debe comentar a medida que se escribe el código, esto ayuda a tener un mejor conocimiento de lo que se está programando.
- Los comentarios deben describir lo que está pasando, cómo se está haciendo, qué significan los parámetros, qué variables se usan y cuáles son modificadas. Además, deben alertar de cualquier restricción en el código.
- Hay que evitar hacer comentarios obvios.
- Es preferible no comentar a escribir comentarios confusos y desactualizados.
- Evitar encerrar los comentarios en cajas dibujadas con asteriscos o cualquier otra tipografía especial.

2.4.3 Pruebas

La fase de producción requiere de pruebas adicionales y revisiones de rendimiento antes de que el sistema sea trasladado al entorno del cliente.

Las pruebas de software son un elemento importante en la calidad del sistema. En este proceso se ejecutan pruebas dirigidas a componentes del software o al sistema en su totalidad, con el objetivo de medir el grado en que este cumple con los requerimientos. Desde que comenzó el desarrollo del sistema de gestión de menú en

los comedores de la Universidad de Las Tunas, este fue sometido a una serie de pruebas para garantizar su correcto funcionamiento.

Las pruebas realizadas al sistema fueron las pruebas de validación y de caja negra. Las primeras se concentran en las acciones visibles para el usuario y en la salida del sistema. La validación se alcanza cuando el software funciona de tal manera que satisface las expectativas del cliente, es decir, cuando es probado cada uno de los requisitos funcionales.

Dentro de las pruebas del nivel de validación se realizan pruebas de tipo beta para descubrir errores que solo el usuario final puede detectar.

Las pruebas beta se realizan en uno o más sitios del usuario final. Por lo general el desarrollador no está presente. Por lo tanto, la prueba beta es una aplicación "en vivo" del software en un ambiente que no puede controlar el desarrollador. El cliente registra todos los problemas que se encuentran durante la prueba beta y los reporta al desarrollador periódicamente.

A continuación se muestran algunas de las pruebas propuestas a realizarse en las diferentes iteraciones definidas.

Tabla 8. Caso de Prueba Iniciar Sesión

Caso de Prueba Iniciar Sesión	
Código: HU1_P01	Historia de usuario: 1
Nombre: Iniciar Sesión	
Descripción: Prueba para la funcionalidad de autenticar un usuario en la aplicación.	
Condiciones de ejecución: Solo los usuarios registrados tienen permiso de autenticarse en el sistema.	
Entrada/ Pasos de Ejecución: El sistema verifica en la base de datos que el usuario existe y que la contraseña para este es correcta.	
Resultado Esperado: El sistema debe mostrar opciones y menús a los cuales el usuario autenticado tiene permiso. En caso de que el usuario ignore algunos de los campos de carácter obligatorio (dígase nombre de usuario o contraseña) o introduzca datos incorrectos se debe informar sobre los errores en los que pudo incurrir, además de comunicarle que no se pudo completar el proceso de inicio de sesión.	
Evaluación de la Prueba: Prueba satisfactoria.	

Tabla 9. Caso de Prueba Inserción de Datos

Caso de Prueba Inserción de Datos	
Código: HU2_P1	Historia de usuario: 2
Nombre: Inserción correcta de datos de un usuario al sistema.	
Descripción: Prueba para la funcionalidad de insertar datos de un usuario correctamente al sistema.	
Condiciones de ejecución: Deben llenarse todos los datos que presenta la tabla, en caso de no llevar ningún valor, se escoge la opción ninguno.	
Entrada/ Pasos de Ejecución: El sistema verifica que todos los datos insertados por el usuario sean correctos.	
Resultado Esperado: En caso de existir algún campo vacío, el sistema mostrará un mensaje de error. En caso contrario, los datos serán guardados en la base de datos.	
Evaluación de la Prueba: Prueba satisfactoria.	

Las pruebas de caja negra, también llamadas pruebas de comportamiento, se enfocan en los requerimientos funcionales del software.

Elas intentan encontrar errores en las categorías siguientes:

1. Funciones incorrectas o faltantes.
2. Errores de interfaz.
3. Errores en la estructura de datos.
4. Errores de comportamiento o rendimiento.

5. Errores de inicialización y terminación.

Caso de prueba para el HU Registro de Usuarios

Caso de Prueba para el HU Registro de Usuarios el cual acepta como parámetros:

Usuario: Valor alfa, campo obligatorio

Nombre: Valor alfa, campo obligatorio

Apellidos: Valor alfa, campo obligatorio

Correo: Valor alfa, campo obligatorio

Contraseña: Valor alfa, campo obligatorio

Rol: Valor alfa, campo obligatorio

Tipo: Valor alfa, campo obligatorio

Clases de Equivalencia

1. Si escribe todos los datos correctamente
2. Si hay uno de los parámetros obligatorios en blanco
3. Si se escribe un correo que no siga el patrón de correo electrónico.
4. La contraseña no es segura.
5. La confirmación de la contraseña no es igual a la contraseña.

En las siguientes tablas se describen los casos de prueba realizados para verificar el correcto funcionamiento del sistema.

Caso de Prueba #1

Caso de Prueba # 1: Comprobar que se registre un usuario donde todos los datos son correctos

Descripción: Se intentará registrar un usuario con todos los datos correctos.
--

Entrada: Usuario: “yami”, Nombre: “Yamilet”, Apellidos: “Yero Hechavarría”, Correo: “yami@gmail.com”, Contraseña: “GmailTest*2025”, Confirmar contraseña: “GmailTest*2025”, Rol: “Administrador”, Tipo: “Admin”.
Resultado: El usuario se registra satisfactoriamente y navega hasta la pantalla de verificación de cuenta.
Condiciones: -
Evaluación de Prueba: Satisfactoria

Caso de Prueba #2

Caso de Prueba # 2: Comprobar que no deje registrar un usuario cuando se deja un parámetro en blanco
Descripción: Se intentará registrar un usuario con parámetros en blanco.
Entrada: Usuario: “yami”, Nombre: “Yamilet”, Apellidos: “ ”, Correo: “yami@gmail.com”, Contraseña: “GmailTest*2025”, Confirmar contraseña: “GmailTest*2025”, Rol: “Administrador”, Tipo: “Admin”.
Resultado: No deja registrar el usuario y muestra mensaje al lado del campo en blanco: “Este campo es obligatorio”.
Condiciones: -
Evaluación de Prueba: Satisfactoria

Caso de Prueba #3

Caso de Prueba # 3: Comprobar que se registre un usuario escribiendo un correo electrónico que no siga con el patrón adecuado.
Descripción: Se intentará registrar un usuario con un correo electrónico con patrón incorrecto.
Entrada: Usuario: “yami”, Nombre: “Yamilet”, Apellidos: “ Yero Hechavarría ”, Correo: “yami gmail.com ”, Contraseña: “GmailTest*2025”, Confirmar contraseña: “GmailTest*2025”, Rol: “Administrador”, Tipo: “Admin”.
Resultado: No deja insertar el usuario y muestra mensaje al lado del parámetro que no cumple con el patrón adecuado: “Estos datos no corresponden con un correo electrónico”.
Condiciones: -
Evaluación de Prueba: Satisfactoria

Caso de Prueba #4

Caso de Prueba # 4: Comprobar que se registre un usuario con una contraseña no segura
Descripción: Se intentará registrar un usuario con una contraseña no segura.
Entrada: Usuario: “yami”, Nombre: “Yamilet”, Apellidos: “Yero Hechavarría”, Correo: “yami @gmail.com ”, Contraseña: “123”, Confirmar contraseña: “123”, Rol: “Administrador”, Tipo: “Admin”.
Resultado: No deja registrar el usuario y muestra mensaje al lado del

parámetro incorrecto: “La contraseña debe tener al menos 8 caracteres y contener mayúsculas, caracteres especiales y números”.
Condiciones: -
Evaluación de Prueba: Satisfactoria

Caso de Prueba #5

Caso de Prueba # 5: Comprobar que se registre un usuario cuando la Contraseña y la confirmación de la contraseña no coinciden.
Descripción: Se intentará insertar un usuario con la confirmación de contraseña distinta a la contraseña.
Entrada: Usuario: “yami”, Nombre: “Yamilet”, Apellidos: “Yero Hechavarría”, Correo: “yami@gmail.com”, Contraseña: “GmailTest*2025”, Confirmar contraseña: “GmailTestWrong*2025”, Rol: “Administrador”, Tipo: “Admin”.
Resultado: No deja registrar el usuario y muestra mensaje al lado del parámetro incorrecto: “La contraseña y la confirmación de la contraseña no coinciden”.
Condiciones: -
Evaluación de Prueba: Satisfactoria

2.4.4 Análisis de los resultados de las pruebas

Para evaluar la calidad del sistema informático de la presente investigación se le realizaron pruebas a las funcionalidades, obteniéndose los siguientes resultados:

- Se realizaron un total de 3 iteraciones de pruebas para comprobar su correcto funcionamiento.
- En la primera iteración se realizaron las pruebas a 9 HU detectándose los siguientes errores: 2 de ortografía, 5 de usabilidad y 6 de validación.
- En la segunda iteración se realizaron las pruebas a 6 HU, se corrigieron los errores detectados en la primera iteración y se detectaron 3 errores de ortografía, 6 de validación y 5 de usabilidad.
- En la tercera iteración se corrigieron los errores detectados en la segunda iteración y se le aplicaron pruebas a las restantes 4 HU detectándose 1 error de usabilidad y 3 de validación.
- Las pruebas realizadas garantizaron la correcta validación del sistema informático.
- La aplicación posee una interfaz agradable y fácil para el usuario.
- Los usuarios solo tienen acceso a la información que les compete de acuerdo con el rol que desempeñan en la entidad.

En la siguiente figura se muestran dichos resultados por iteraciones.

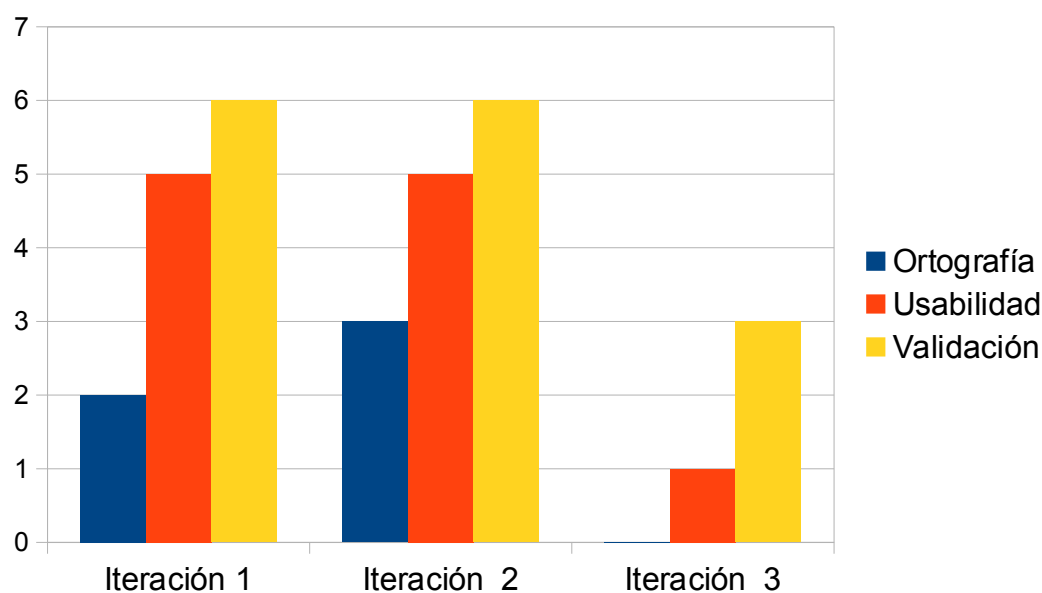


Figura 3. Resultados de las pruebas realizadas al sistema

Conclusiones del capítulo

En el desarrollo de este capítulo, se crearon principales artefactos de Scrum en todas sus fases, donde el cliente a través de las historias de usuario da una breve descripción de lo que desea que realice la aplicación. Se creó el plan de iteraciones agrupando las historias de usuario según la prioridad asignada a cada una y se realizó el plan de entregas donde se estima el tiempo de desarrollo en semanas para cada una de las iteraciones.

CAPÍTULO III. IMPLEMENTACIÓN DEL SISTEMA.

En este capítulo se tratarán elementos que son necesarios para dar una amplia perspectiva del objetivo que se persigue con la implementación del sistema para la gestión de menú en los comedores de la Universidad de Las Tunas. Se presenta información del diseño de la interfaz del software, las formas generales y principios en que se basa el sistema de ayuda, el sistema de protección y seguridad, así como el tratamiento de errores.

3.1 Diseño de las interfaces de usuario.

El Diseño de Interfaz de Usuario adquiere una elevada importancia en la usabilidad, la experiencia de usuario y la calidad del software. En la vida cotidiana interactuamos constantemente con Interfaces Gráficas de Usuario (GUI por su nombre en inglés Graphical User Interface); no solo al usar una computadora sino también en objetos de uso diario como el celular, el cajero automático, etc. entre muchas otras cosas. La interfaz tiene un papel fundamental para que el producto sea o no competitivo. El producto no será exitoso si el usuario no consigue concretar una acción, o no entiende la secuencia de pasos a seguir, o cuando no encuentra con facilidad cómo concretar la acción que necesita o cuando no considera atractivo el diseño de la aplicación que está utilizando. El diseño de la GUI suele considerarse como tarea secundaria, sin embargo al analizar por qué algunas aplicaciones fracasan, se evidencia que el diseño de la GUI tiene una gran influencia en esto [26].

En la figura 4 se puede apreciar la primera interfaz a cargar por el sistema, que será la de autenticación.

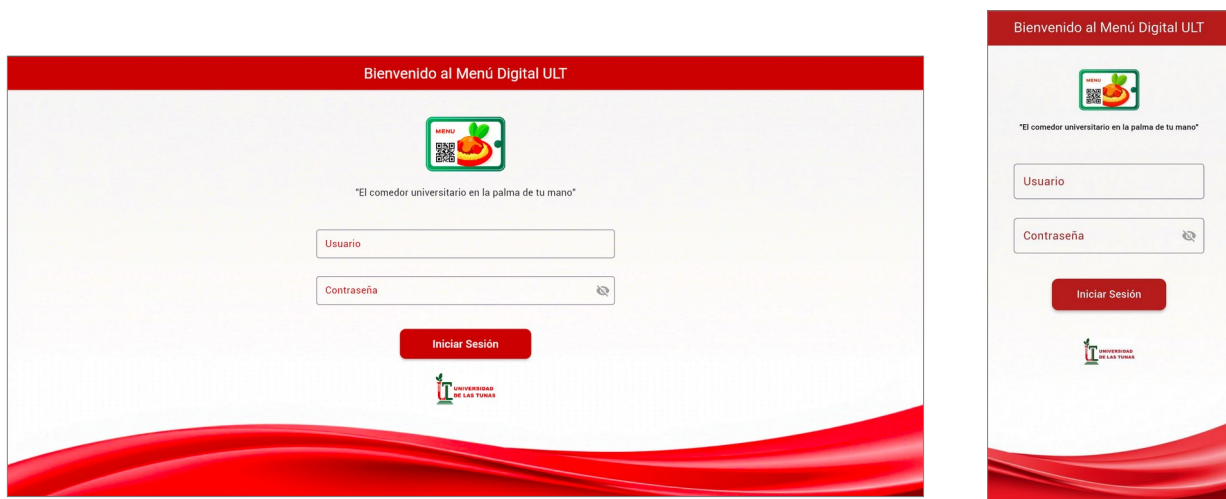


Figura 4. Interfaz de autenticación

En dependencia del rol del usuario que se autentique, la página principal del sistema se adaptará al alcance de sus permisos. El rol de Usuario solo tendrá acceso a la reserva de tickets, es decir, esa será su página principal una vez autenticado. En cambio los demás roles navegarán hacia la página Opciones, donde según su rol, tendrán acceso a los diferentes módulos.

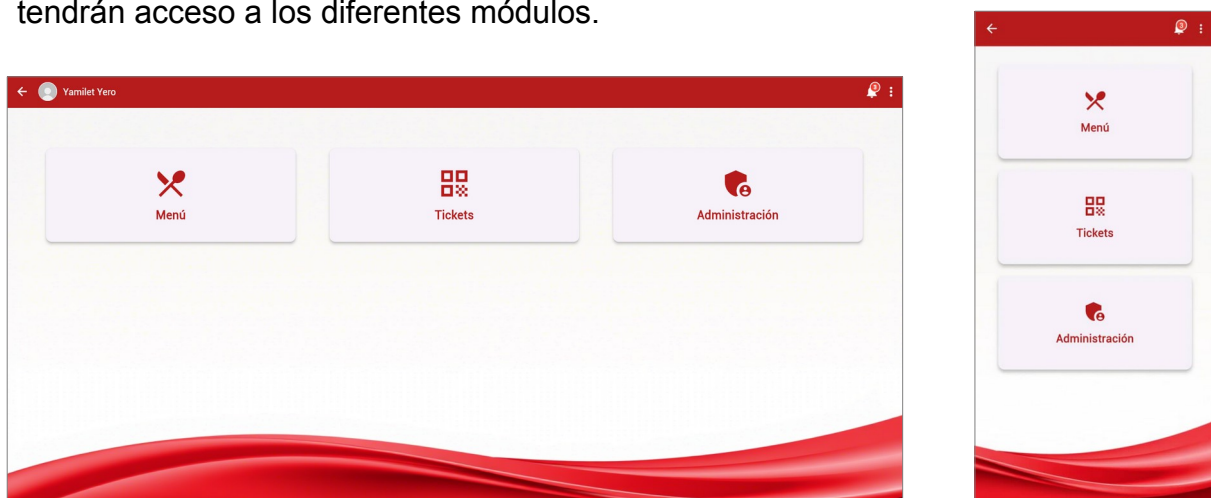


Figura 5. Interfaz Principal

En la interfaz principal (Figura 5) se pueden apreciar módulos que facilitan los accesos según los roles del sistema.

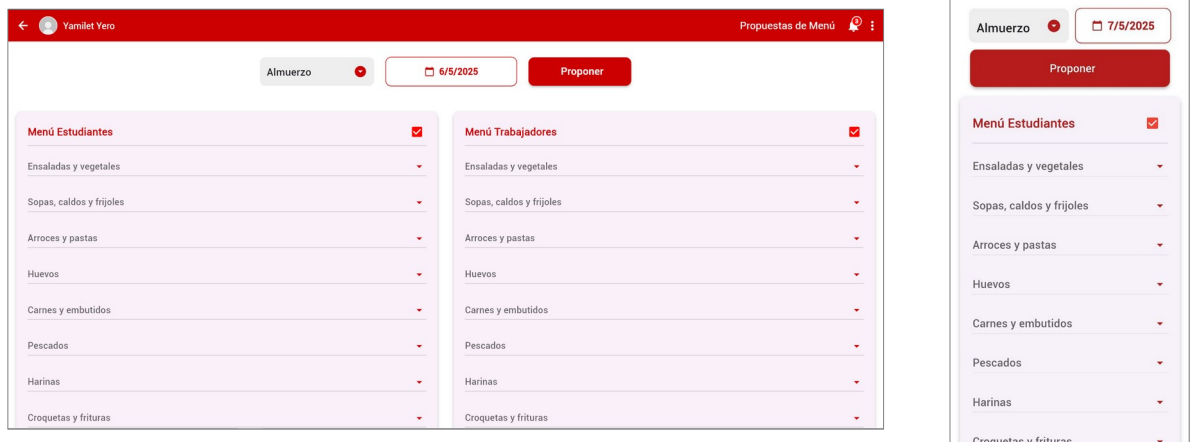


Figura 6. Interfaz de Propuestas de Menú

En esta interfaz (Figura 6) se aprecia la elaboración de las propuestas de menú para estudiantes y trabajadores.

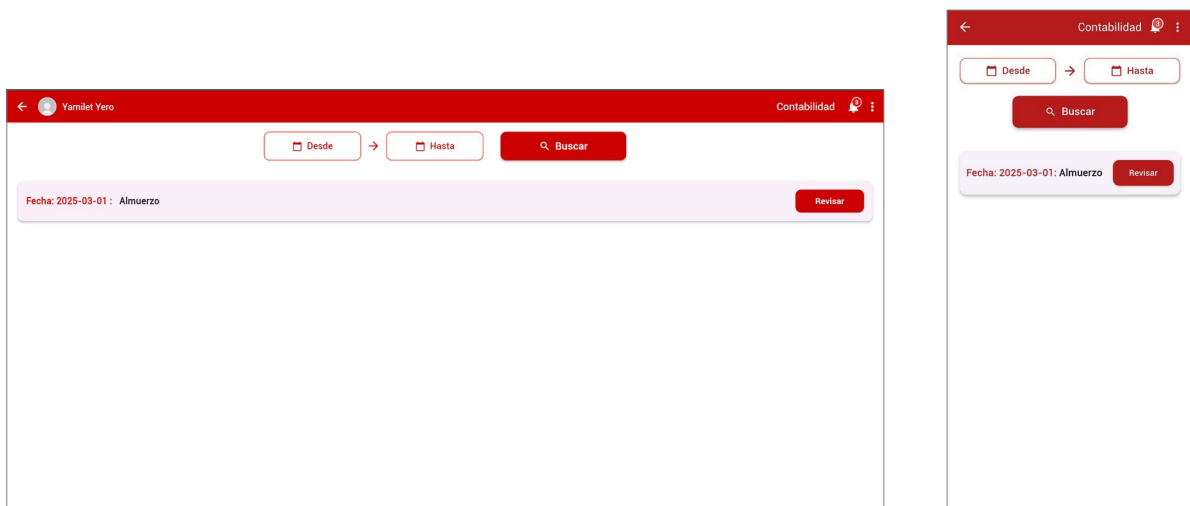


Figura 7. Interfaz de Aprobación de Menú (Lista de propuestas)

En la interfaz Aprobar Menú (Figura 7) el contador hace revisión de las propuestas elaboradas por la Comisión y decide si estas son aprobadas o no y si precisan de algún ajuste de precios. Estos detalles se muestran a continuación en la figura 7.

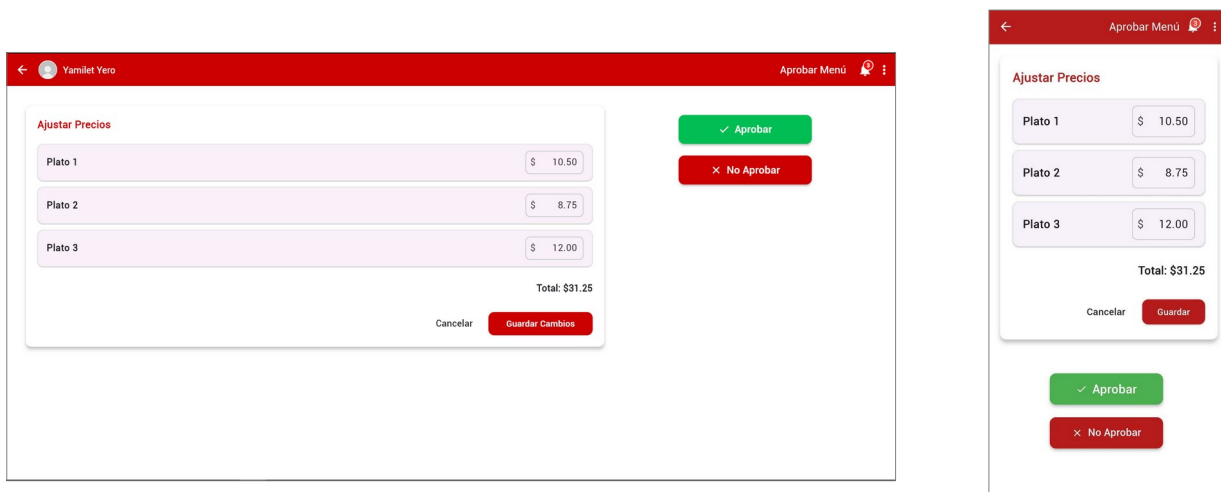


Figura 8. Interfaz de Reserva de Tickets

En la interfaz de Reserva de Tickets (Figura 8) se seleccionan el comedor, horario de menú, fecha y se realiza una búsqueda de los menús disponibles, escogiendo de estos los platos deseados por el usuario, conformando así un menú personalizado.

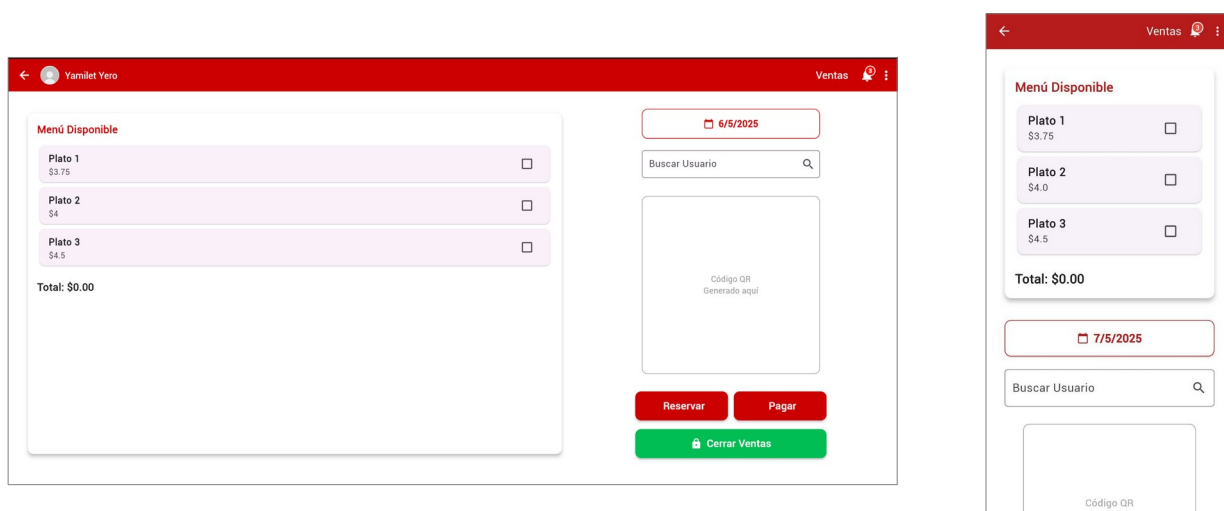


Figura 9. Interfaz de Ventas

En la interfaz de Ventas (Figura 9) se aprecia el procedimiento para la reserva/venta de los tickets en la instalaciones universitarias con un día de antelación. El usuario selecciona los platos a consumir, especifica si es una reserva o una compra y culminado ese proceso se genera un código QR que contiene los datos necesarios para posteriormente poder acceder al comedor a través de su escaneo.

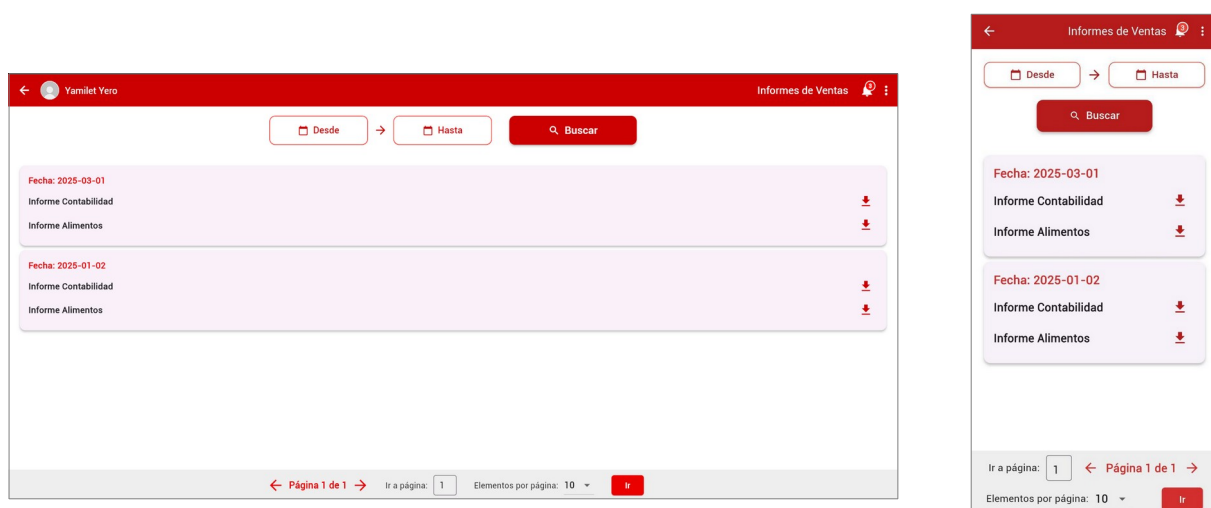


Figura 10. Interfaz de Informes de Ventas

En la interfaz de Informes de Ventas (Figura 10) se pueden apreciar los informes generados al final de cada cierre de ventas, dando a conocer la cantidad de platos vendidos en el informe para el Departamento de Alimentos y la cantidad de tickets foliados con su importe total al Departamento de Contabilidad.

3.2. Protección y seguridad del sistema.

La seguridad informática, con sus siglas en inglés IT security, es la disciplina que se encarga de llevar a cabo las soluciones técnicas de protección de la información. La seguridad informática protege el sistema informático, tratando de asegurar la integridad y la privacidad de la información que contiene. Por lo tanto, podríamos decir, que se trata de implementar medidas técnicas que preservarán las

infraestructuras y de comunicación que soportan la operación de una empresa, es decir, el hardware y el software empleados por la empresa [27].

El análisis del nivel de seguridad de los roles y privilegios, se realiza con el fin de garantizar la adecuada configuración del sistema de gestión a través de roles y moderar los riesgos que puedan afectar la integridad, disponibilidad y confidencialidad de la información, a través de la autenticación al sistema revisa que los datos introducidos por el usuario sean correctos usuario y contraseña en dependencia del usuario autenticado se accede a la interfaz destinada a cada uno.

La gestión de trazas en los sistemas informáticos es un proceso fundamental para garantizar la seguridad. Es de gran importancia conocer los accesos realizados, u otros eventos que permiten determinar el comportamiento de un sistema en un período de tiempo. Las trazas se generan en diferentes formatos lo que hace difícil su procesamiento. La gestión de trazas requiere de sistemas que procesen y normalicen la gran variedad de formatos existentes. También es fundamental definir mecanismos de transporte, planeación y ejecución, sistemas de almacenamiento eficientes en cuanto la utilización de espacio y herramientas para la búsqueda y detección de patrones [28].

En la figura 11 se muestra la actividad de los usuarios:

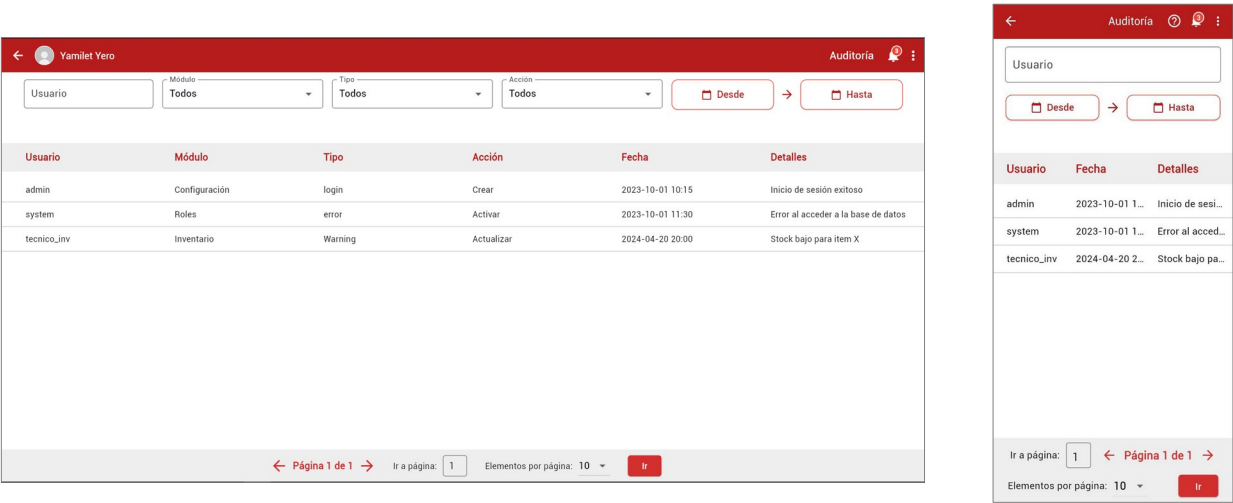
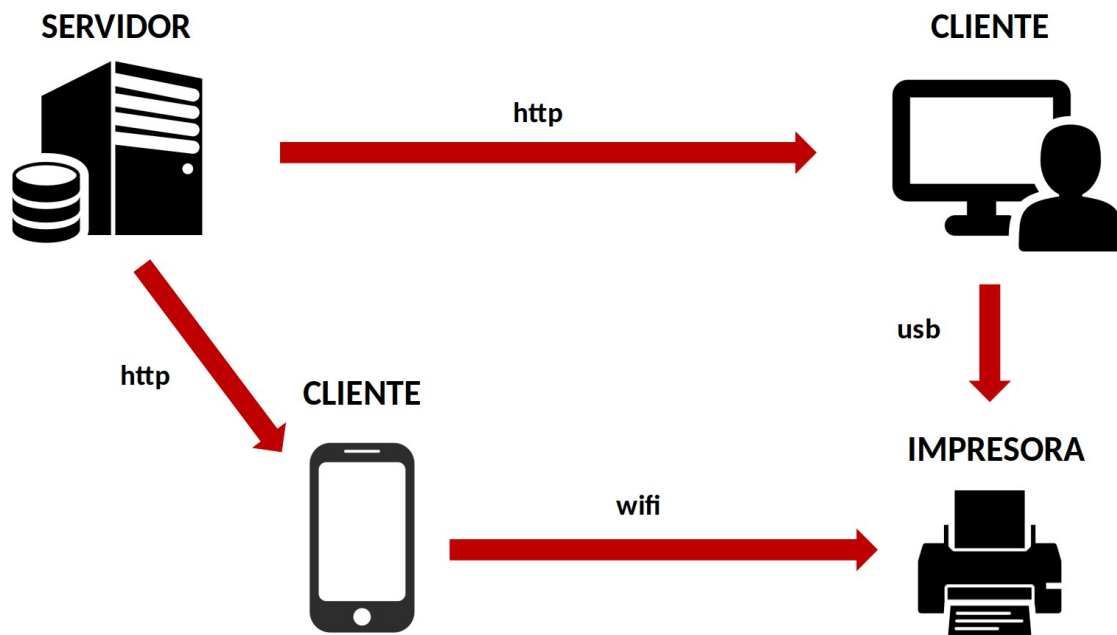


Figura 11. Interfaz de Auditoría

3.2.1 Distribución física del sistema

La arquitectura física del sistema se centra en un modelo cliente-servidor distribuido, donde un servidor central gestiona los datos y la lógica principal, comunicándose con diversos tipos de dispositivos cliente a través del protocolo HTTP. Estos clientes incluyen tanto estaciones de trabajo de escritorio como dispositivos móviles, cada uno con la capacidad de interactuar con el servidor para acceder a los servicios del sistema. Adicionalmente, el sistema contempla la funcionalidad de impresión, permitiendo que los clientes de escritorio se conecten a impresoras mediante USB, mientras que los clientes móviles pueden utilizar conexiones Wi-Fi para el mismo fin, demostrando flexibilidad en el acceso a periféricos.



3.3 Forma general y principios en que se basa el sistema de ayuda

El sistema cuenta con un manual de usuario (Figura 12), el mismo tiene como objetivo facilitar la tarea de conocimiento, uso y aprendizaje del sistema desarrollado. Contiene información acerca de todas las operaciones básicas que el sistema ofrece, así como capturas de pantallas útiles para el seguimiento de la explicación. Este manual es accesible desde la cabecera de la aplicación, a través del ícono Ayuda (Manual de usuarios).

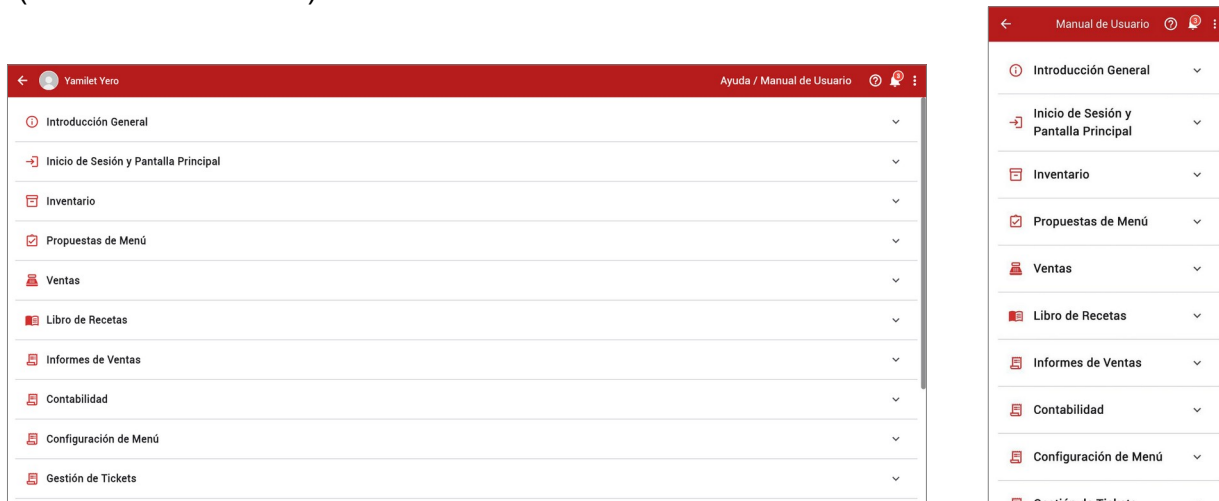


Figura 12. Manual de Usuarios

3.4 Tratamiento de errores.

El manejo de errores se refiere a los procedimientos de respuesta y recuperación de las condiciones de error presentes en una aplicación de software. En otras palabras, es el proceso compuesto por anticipación, detección y resolución de errores de aplicación, errores de programación o errores de comunicación. El manejo de errores ayuda a mantener el flujo normal de ejecución del programa. El manejo de errores ayuda a manejar los errores de hardware y software con gracia y ayuda a reanudar la ejecución cuando se interrumpe. Cuando se trata de manejo de errores en software, el programador desarrolla los códigos necesarios para manejar errores o utiliza herramientas de software para manejar los errores. En los casos en que los errores no se pueden clasificar, el manejo de errores generalmente se realiza con la devolución de códigos de error especiales.

En la realización del software los errores se manejan mediante el relleno de formularios. Teniendo en cuenta que sean completados con el tipo de datos que se pide, en el momento de insertar un usuario no puede poner en el campo Nombre un número, en el campo Nombre el dato esta predefinido.

3.5 Importancia del sistema

Este sistema tiene como objetivo impulsar la eficiencia en el proceso de gestión de menú en los comedores de la Universidad de Las Tunas. Administra la información relevante, ofrece un acceso ágil y ordenado a los datos con la informatización de los procesos operativos y proporciona información clave y útil para la toma de decisiones. Al mismo tiempo, facilita el control de todos los elementos necesarios para que el proceso se realice con la calidad requerida, distanciándose de los métodos convencionales (documentación impresa, carpetas y archivos físicos) mediante su integración con la red. Suprime la incidencia de errores de cálculo al determinar los costos de los menús y las transacciones de venta. Permite la recopilación, el resguardo y el procesamiento de la información de forma eficiente y efectiva, lo que reduce los gastos del procedimiento y el tiempo de ejecución. Los informes se elaboran en un lapso inferior y se unifica la información, optimizando de esta manera la protección en el acceso a los documentos generados durante el procedimiento.

Conclusiones del capítulo

En este capítulo se trataron los aspectos concernientes a la implementación del sistema, el diseño de la interfaz de manera general, los fundamentos de protección y seguridad informática, así como el sistema de ayuda creado como herramienta de soporte para su utilización. El sistema desarrollado permite gestionar de forma eficiente la información del menú de alimentación en los comedores de la Universidad de Las Tunas, demostrando ser efectivo y aportando a la optimización de la calidad de los servicios prestados.

CONCLUSIONES GENERALES

1. El estudio realizado al proceso de gestión de menú en los comedores de la Universidad de Las Tunas permitió identificar dificultades en la gestión de menú que limitan la calidad de los servicios en los comedores de la Universidad de Las Tunas.
2. Al analizar sistemas similares creados para la gestión de menú tanto a nivel foráneo como nacional se pudo comprobar que estos no cumplen los requisitos necesarios para introducir su uso en los comedores de la Universidad de Las Tunas.
3. El análisis realizado de las metodologías, lenguajes y herramientas existentes, permitió seleccionar como metodologías de desarrollo Scrum, los frameworks Spring Boot 3 y Flutter, Java y Dart como lenguajes de programación y PostgreSQL como gestor de base de datos, los que permitieron implementar el sistema informático.
4. Se realizó el diseño e implementación del sistema informático, utilizando los diferentes patrones y estándares de codificación, que permitieron gestionar de forma eficiente la gestión de menú en la Universidad de Las Tunas.
5. Al realizar las pruebas a las funcionalidades del sistema, se pudo constatar el alto grado de coincidencia entre las expectativas del cliente y la funcionalidad del software.

RECOMENDACIONES

REFERENCIAS BIBLIOGRÁFICAS

- [1]. A. C. de Noticias, «Impulsa universidad central la producción de alimentos», ACN. Accedido: 16 de mayo de 2025. [En línea]. Disponible en: <https://www.acn.cu/cuba/impulsa-universidad-central-la-produccion-de-alimentos>
- [2].....«12 Efficiency and Profitability Benefits of QR Code Menus – Restaurant365», <https://www.restaurant365.com/>. Accedido: 16 de mayo de 2025. [En línea]. Disponible en: <https://www.restaurant365.com/blog/12-efficiency-and-profitability-benefits-of-qr-code-menus/>
- [3]...K. A. Vargas Guzmán y D. M. León Castañeda, «Implementación de Código QR como Método de Codificación, para Sistema de Inventario a Través de Un Aplicativo Móvil y Servicios Web», jul. 2017, Accedido: 16 de mayo de 2025. [En línea]. Disponible en: <http://hdl.handle.net/11349/5967>
- [4].....«OPTIMIZACIÓN DEL PROCESO DE ELABORACIÓN DE RACIONES EN UN SERVICIO DE ALIMENTACIÓN COLECTIVA».
- [5].....S. M. Gil y I. Hoyos, «91 PUBLICATIONS 1,535 CITATIONS SEE PROFILE».
- [6]..«Top 9 mejores Software de Alimentación [guía 2025]». Accedido: 10 de mayo de 2025. [En línea]. Disponible en: <https://www.softwaredoit.es/software-industrial/software-alimentacion-alimentaria.html>
- [7].....«Versat Sarasola – Datazucar». Accedido: 16 de mayo de 2025. [En línea]. Disponible en: https://www.datazucar.cu/?page_id=1739
- [8].Y. A. L. Montoya y A. P. Céspedes, «Sistema informático para la gestión del menú de alimentación en el Comedor Industria de la Empresa de Servicios a la Agroindustria Azucarera Colombia.».
- [9].....E. Gabriel, «Metodologías de desarrollo de software».

[10].....C. Ignacio y V. Paola, «Metodologías actuales de desarrollo de software», n.º 2015, 2015.

[11].....«¿En qué consiste Scrum? - Explicación sobre la metodología Scrum - AWS», Amazon Web Services, Inc. Accedido: 10 de mayo de 2025. [En línea]. Disponible en: <https://aws.amazon.com/es/what-is/scrum/>

[12].....E. Sandoval, «Frameworks: Marcos de trabajo para programadores», Ebac. Accedido: 16 de mayo de 2025. [En línea]. Disponible en: <https://ebac.mx/blog/frameworks>

[13].....«Spring Boot :: Spring Boot». Accedido: 10 de mayo de 2025. [En línea]. Disponible en: <https://docs.spring.io/spring-boot/>

[14].....«¿Qué es Flutter? - Explicación de la aplicación Flutter - AWS», Amazon Web Services, Inc. Accedido: 10 de mayo de 2025. [En línea]. Disponible en: <https://aws.amazon.com/es/what-is/flutter/>

[15]....«Flutter documentation». Accedido: 10 de mayo de 2025. [En línea]. Disponible en: <https://docs.flutter.dev>

[16]. .«Java Documentation - Get Started». Accedido: 10 de mayo de 2025. [En línea]. Disponible en: <https://docs.oracle.com/en/java/>

[17].....«Dart documentation | Dart». Accedido: 10 de mayo de 2025. [En línea]. Disponible en: <https://dart.dev/docs>

[18].....«PostgreSQL: Documentation». Accedido: 10 de mayo de 2025. [En línea]. Disponible en: <https://www.postgresql.org/docs/>

[19].....E. Gamma, *Patrones de diseño: elementos de software orientado a objetos reutilizable*. Pearson Educación, 2002.

[20]. .«MVC - Glosario de MDN Web Docs: Definiciones de términos relacionados con la Web | MDN». Accedido: 10 de mayo de 2025. [En línea]. Disponible en: <https://developer.mozilla.org/es/docs/Glossary/MVC>

[21]...«Introducción a Android Studio», Android Developers. Accedido: 20 de mayo de 2025. [En línea]. Disponible en: <https://developer.android.com/studio/intro?hl=es-419>

[22]. «IntelliJ IDEA overview | IntelliJ IDEA», IntelliJ IDEA Help. Accedido: 20 de mayo de 2025. [En línea]. Disponible en: <https://www.jetbrains.com/help/idea/discover-intellij-idea.html>

[23]....«Apidog An integrated platform for API design, debugging, development, mock, and testing», apidog. Accedido: 20 de mayo de 2025. [En línea]. Disponible en: <https://apidog.com/>

[24].....«<https://quantumit.com.co/wp-content/uploads/2022/06/guia-scrum.pdf>». Accedido: 16 de mayo de 2025. [En línea]. Disponible en: <https://quantumit.com.co/wp-content/uploads/2022/06/guia-scrum.pdf>

[25].C. Rafael José y N. B. Wilson, *Diseño de bases de datos*. Universidad del Norte, 2017.

[26] S. M. Labrada, «Principios del proceso de diseño de interfaz de usuario», *Revista Cubana de Transformación Digital*, vol. 1, n.º 3, Art. n.º 3, dic. 2020.

[27].....J. A. Figueroa-Suárez, R. F. Rodríguez-Andrade, C. C. Bone-Obando, y J. A. Saltos-Gómez, «La seguridad informática y la seguridad de la información», *Polo del Conocimiento*, vol. 2, n.º 12, Art. n.º 12, 2017, doi: 10.23857/pc.v2i12.420.

[28].....J. Porven Rubier y R. Montesino Perurena, «Marco de trabajo para la gestión centralizada de trazas de seguridad usando herramientas de código abierto», *Revista Cubana de Ciencias Informáticas*, vol. 9, n.º 3, pp. 18-32, sep. 2015.

ANEXOS

Anexo 1. Resumen diario de la existencia de alimentos



MINISTERIO DE EDUCACIÓN SUPERIOR
UNIVERSIDAD DE LAS TUNAS, CUBA

Fecha: _____

Resumen diario de la existencia de alimento

Platos Fuerte

Productos	U/M	P. Tey	Lenin	Total
Pollo Congelado	kg			
Masa/Chorizo de res	kg			
Picadil/Cond. de res	kg			
Huevos frescos	kg			
Pic/Condim Pescado	kg			
Pescado /Mar	kg			
Carne de Ovejo	kg			
Muslo de pollo	kg			
M/hamburg/pes	kg			
Rueda de tenca	kg			

Granos

Productos	U/M	P. Tey	Lenin	Total
Arroz Importado	kg			
Arroz Criollo	kg			
Chicharo	kg			
Frijol Negro	kg			
Frijol Caupi	kg			
Sal Fina	kg			
Harina Maiz Estudi	kg			
Harina Maiz trabaja	kg			
Azúcar Crudo	kg			
Aceite Refino	kg			

Viandas

Productos	U/M	P. Tey	Lenin	Total
Plát Burro Estudian	kg			
Plát, Burro Obrero	kg			
Boniato	kg			
Calabaza	kg			
Yuca	kg			
Papa consumo				

Ensaladas

Productos	U/M	P. Tey	Lenin	Total
Encurtido vegetal	kg			
Tomate	kg			
Col	kg			
Rábano	kg			
Zanahoria	kg			
Vinagre	Lts			
Piña	kg			

Especies y Condimento

Productos	U/M	P. Tey	Lenin	Total
Pasta Ajo	kg			
Cond. Natur.	kg			
Puré Tomate	kg			
Ají A/Pollo	kg			
Ají Chay	kg			
Cebolla Fresca	kg			
Cebollín	kg			
Cond. Mixto	kg			
Pimienta Encurtido	kg			

Mermeladas, Refrescos y Otros

Productos	U/M	P. Tey	Lenin	Total
Concent(sirope)	lts			
M/mango	kg			
M/guayaba	kg			
Pan suave 50g	U			
Tort Casabe 150 g	U			
Q/fundido	kg			
Yogur natural	Bol			

Elaborado por: _____

Anexo 2. Propuesta de Menú

Propuesta de Menú		Fecha	D	M	A
Desayuno	Almuerzo	Comida	Merienda		
1	1	1	1		
2	2	2	2		
3	3	3	3		
4	4	4	4		
5	5	5	5		
6	6	6	6		
	7	7			
	8	8			
	9	9			
	10	10			
	11	11			

Propuesta de Menú		Fecha	D	M	A
Desayuno	Almuerzo	Comida	Merienda		
1	1	1	1		
2	2	2	2		
3	3	3	3		
4	4	4	4		
5	5	5	5		
6	6	6	6		
	7	7			
	8	8			
	9	9			
	10	10			
	11	11			
	12	12			

Anexo 3. Oferta de Menú

UNIVERSIDAD DE LAS TUNAS OFERTA			
Fecha:			
No.	Nombre del plato	Sal	Aceite
1.			
2.			
3.			
4.			
5.			
6.			
7.			
8.			
9.			
10.			

Código	Producto	Gram	Precio